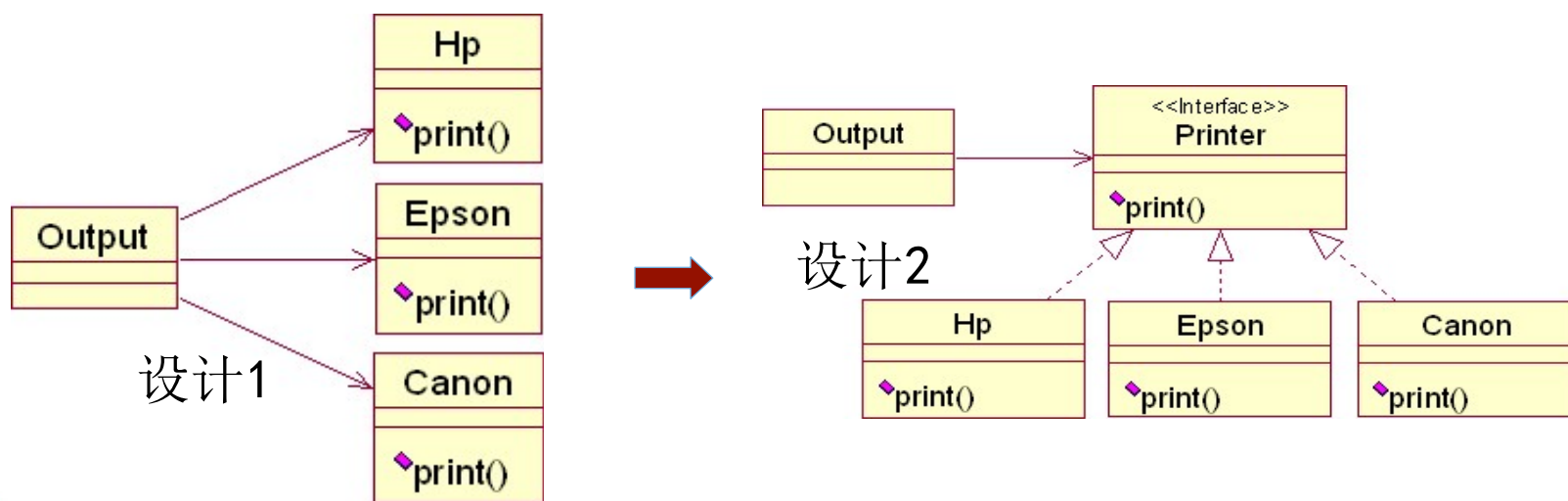


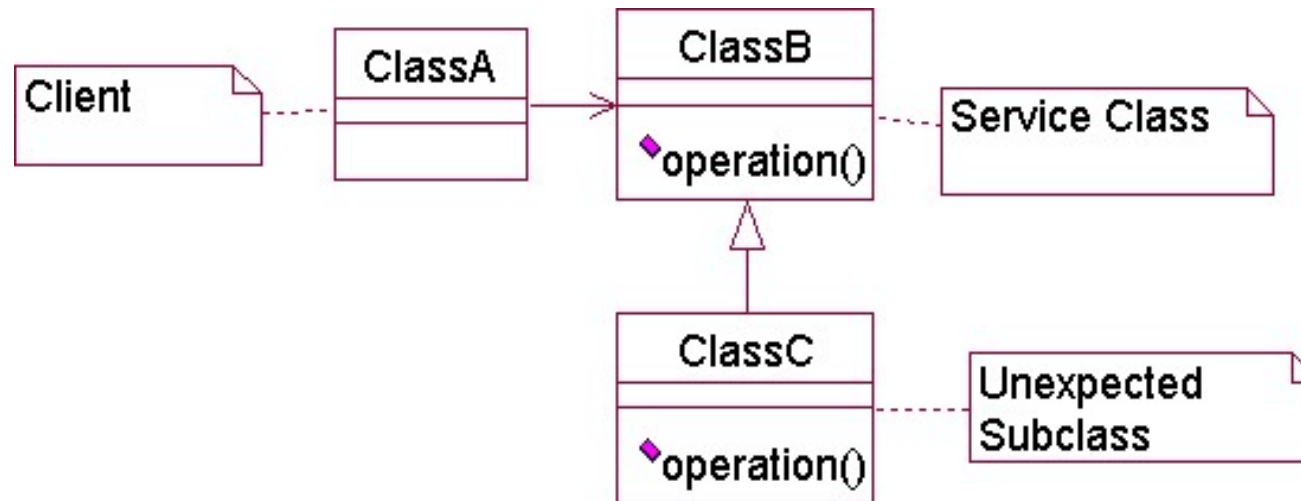
开闭原则(Open/Closed Principle, OCP)

- 最初由Bertrand Meyer提出
- 软件实体在扩展性方面应该是开放的，而在更改性方面应该是封闭的。
- 例：打印输出设计



Liskov Substitution Principle (LSP)

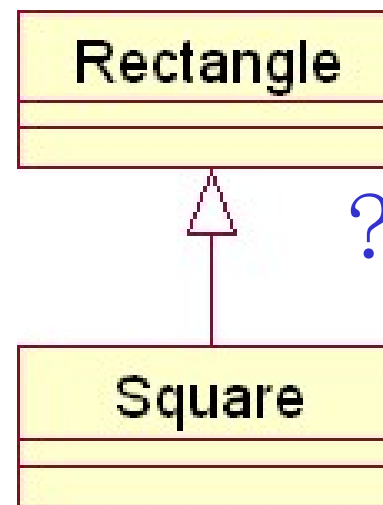
- 最早由Liskov于1987年在OOPSLA会议上提出
- 子类可以替换父类出现在父类能出现的任何地方



违反LSP的例子

```
public class Rectangle
{
    private int topLe5X;
    private int topLe5Y;
    int width;
    int height;

    public void setWidth(int width)
    {
        this.width = width;
    }
    public void setHeight(int height)
    {
        this.height = height;
    }
    public int getWidth()
    {
        return width;
    }
    public int getHeight()
    {
        return height;
    }
}
```

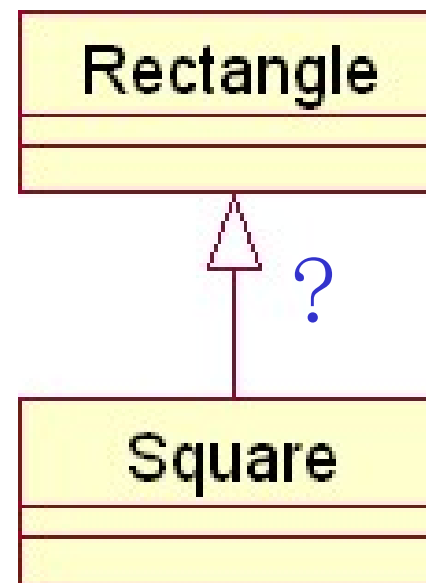


问题：如果将Square作为Rectangle的子类，则如何定义Square类的setWidth和setHeight方法？



解决方法

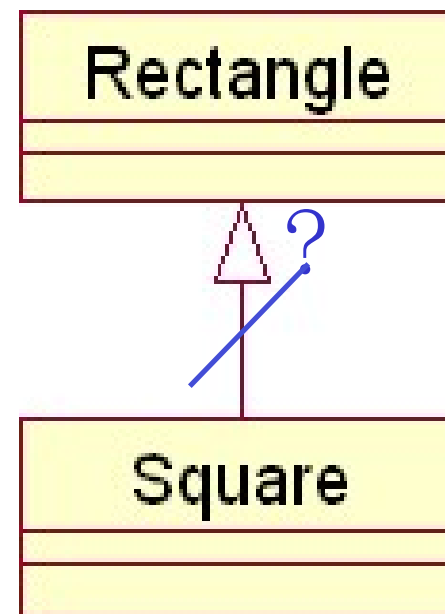
```
public class Square extends Rectangle
{
    public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
    }
    public void setHeight(int height)
    {
        super.setHeight(height);
        super.setWidth(height);
    }
}
```



这种解决方法是否可行？

考虑下面的代码：

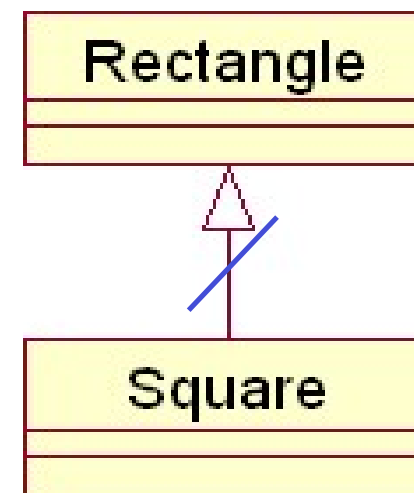
```
public class Test {  
    public static void main(String[] args)  
    {  
        Test t = new Test();  
        Rectangle r = new Rectangle();  
        Square s = new Square();  
        t.g(r);  
        // t.g(s);  
    }  
    private void g(Rectangle r) {  
        r.setWidth(10);  
        r.setHeight(20);  
        assert (r.getWidth()*r.getHeight() == 200);  
    }  
}
```



如果传给方法g的参数是Rectangle类型的对象，则没有问题，如果是Square类型的对象，则出错。

思考题：如何知道子类的行为符合父类的要求？

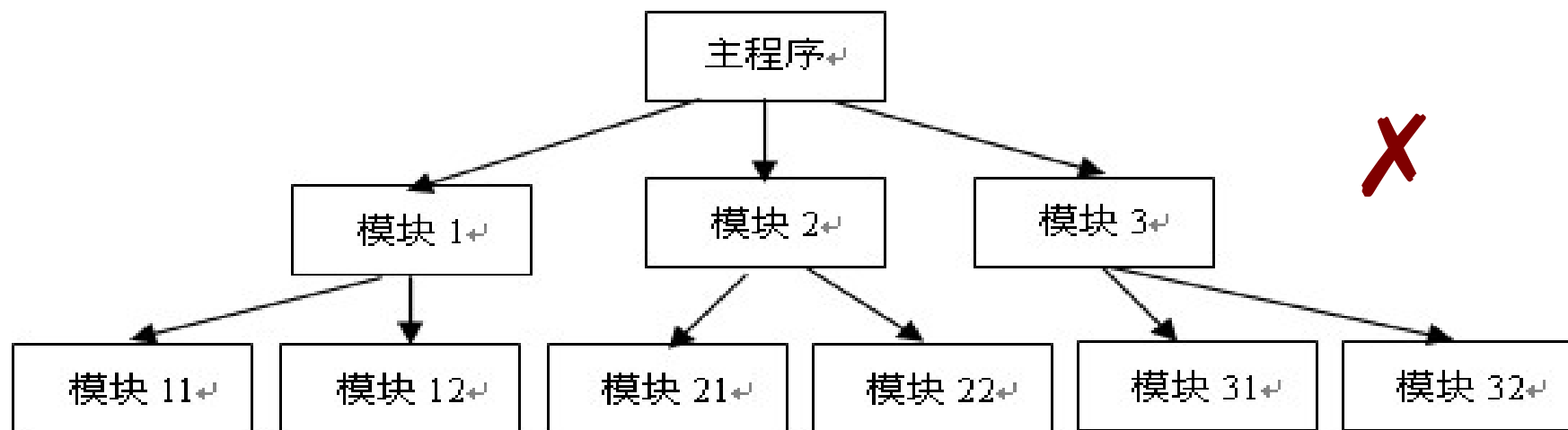
- 契约式设计 (Design by Contract)
- 为了满足Liskov替换原则，设计时要求：
- 子类中方法的前置条件不能强于父类中相应方法的前置条件。
- 子类中方法的后置条件不能弱于父类中相应方法的后置条件。



Liskov 替换原则要求子类宽入严出！

依赖倒置原则 (Dependency Inversion Principle, DIP)

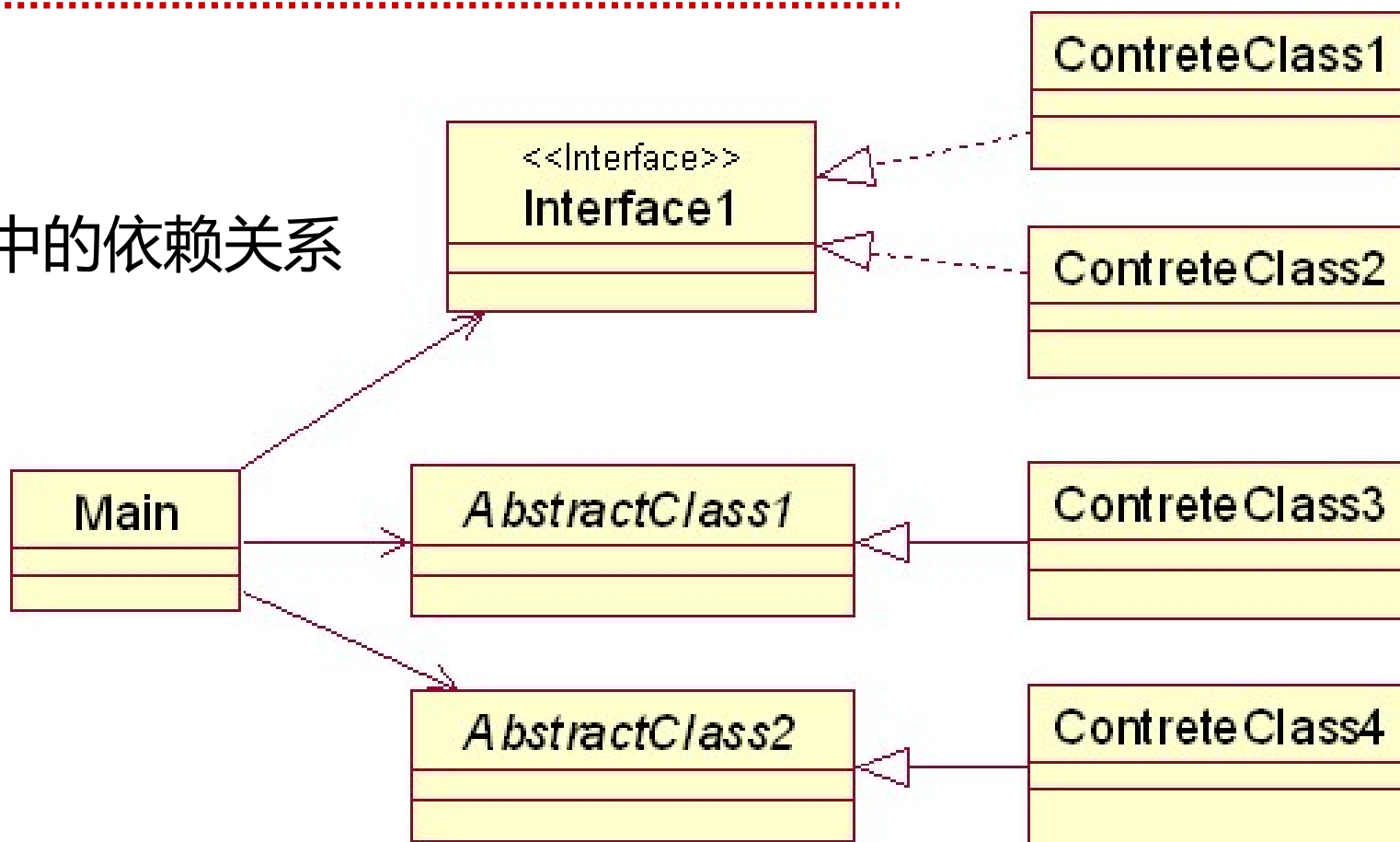
- 依赖倒置原则指的是依赖关系应该是尽量依赖接口(或抽象类), 而不是依赖于具体类。



结构化设计中模块间的依赖关系

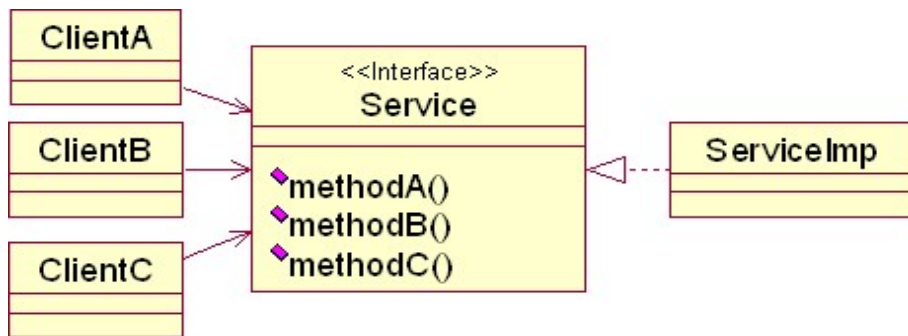
面向对象设计中的依赖关系

OOD中的依赖关系

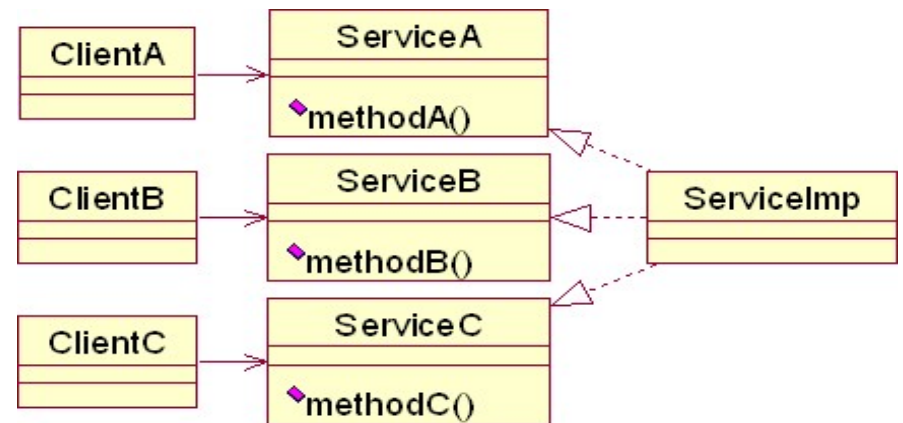


接口分离原则 (Interface Segregation Principle, ISP)

- 在设计时采用多个和特定客户类 (client) 有关的接口要比采用一个通用的接口要好。



使用通用接口的设计



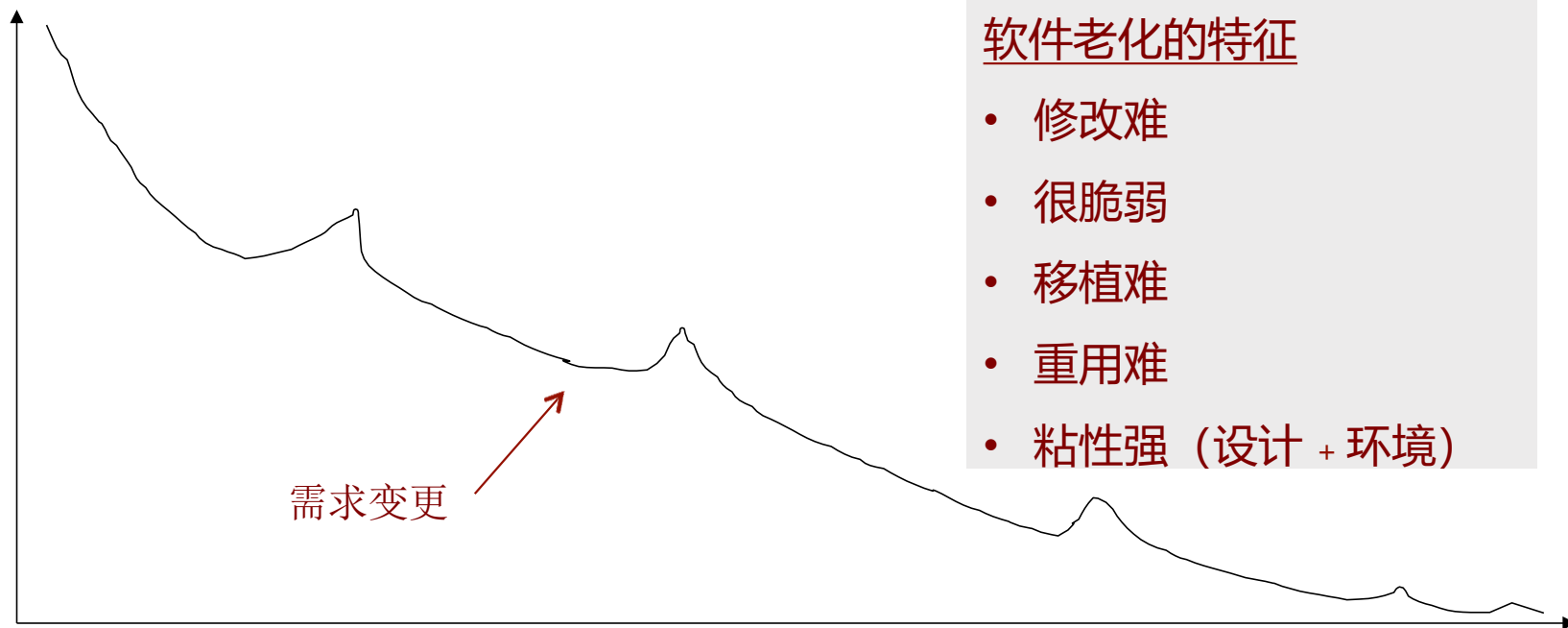
使用分离接口的设计

好的系统设计的特征

- 用户友好
- 易理解
- 可靠
- 可扩展
- 可移植
- 可伸缩
- 可重用

• ...

简单性：实现简单，使用简单，理解简单，维护简单



软件老化的特征

- 修改难
- 很脆弱
- 移植难
- 重用难
- 粘性强 (设计 + 环境)

○○设计时要注意的一些问题

1. 不同类中相似方法的名字应该相同
2. 遵守已有的约定俗成的习惯
3. 尽量减少消息模式的数目。只要可能，就使消息具有一致的模式，以利于理解。
4. 设计简单的类。类的职责要明确，应该从类名就可以较容易地推断出类的用途。
5. 定义简单的操作、方法
6. 定义简单的交互协议
7. 泛化结构的深度要适当
8. 把设计变动的副作用减至最少

《狼人杀》游戏规则：狼人杀游戏通过抽取卡牌将参与游戏的人分为两个对立的阵营：狼人阵营和好人阵营。好人阵营由平民和神职人员组成。

游戏规则：游戏以投票和技能来消灭所有隐藏身份的狼人；狼人则以猎杀和诱导错误投票来消灭所有平民或者神职人员，获得胜利。

每一局狼人杀游戏都将从夜晚开始，每一个夜晚狼人可以睁眼并共同猎杀场上的一位玩家。夜晚之后将会进入白天，白天是所有玩家发言和投票的环节。

每个白天场上存活的玩家将按顺序依次进行发言，在所有玩家发言结束后，法官将会组织一次放逐公投，在放逐公投中每一位玩家都拥有一票的投票权，玩家可以选择把票投给自己心目中所认为的狼人。

最终，得票数最多的玩家将会被放逐出局并留下“遗言”。“遗言”之后游戏将会进入回下一个夜晚，狼人又可以再一次睁眼选择一位玩家进行猎杀，如此“黑夜—白天—黑夜”循环，直到某一方阵营达成了胜利条件，则游戏结束。

以下哪个候选类应该被筛除？

提交

- A 平民
- B 夜晚
- C 狼人杀
- D 遗言



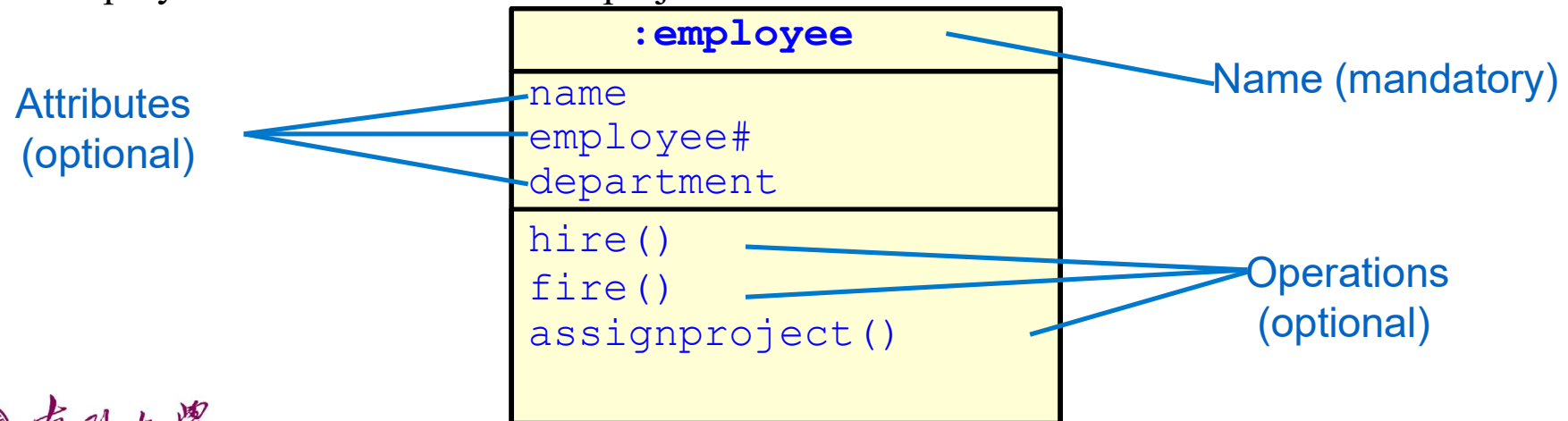
UML类图

UML Class Diagram



什么是类?

- 类是具有以下特征的对象集合
 - 相同性质(attributes)
 - 相同行为(operations)
 - 相同的对象关系
 - 相同语义("semantics")
- 举例
 - **employee**: has a name, employee# and department; an employee is hired, and fired; an employee works in one or more projects



对象

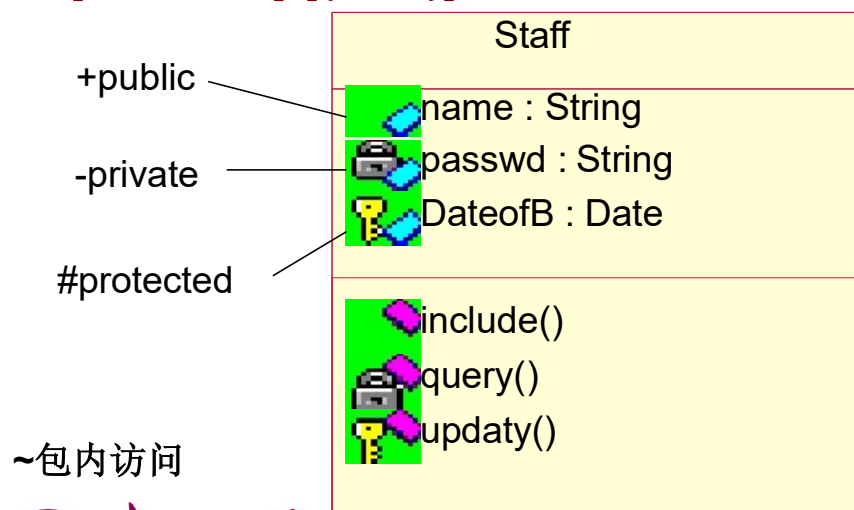
- 对象是类的实例，表示为：

<code>Fred_Bloggs:employee</code>
<code>name: Fred Bloggs</code>
<code>Employee #: 234609234</code>
<code>Department: Marketing</code>

- 两个不同的对象可以有相同的属性取值(正如两个同名的人，或者住在同一栋楼的人)
- 对象与其他对象之间发生关联关系
 - 例如：**Fred_Bloggs:employee** 对象与 **KillerApp:project** 对象相关联 但这个关联关系要定义为类之间的关系 (为什么？)
- 注意将属性划归正确的类
 - 例如：不要将经理姓名和员工编号同时定义为project类的属性 (...为什么？)

类属性定义

- 属性在类图标的属性分隔框中
- 用文字串说明，UML规定属性 的语法为：
- [可见性] 属性名 [:类型] [[多样性 [次序]]]
- [= 初始值] [{约束}]



例：一些属性声明的例子。

+size: Area = (100,100)

#visibility: Boolean = false

+default---size: Rectangle

#maximum---size: Rectangle

—xptr: XwindowPtr

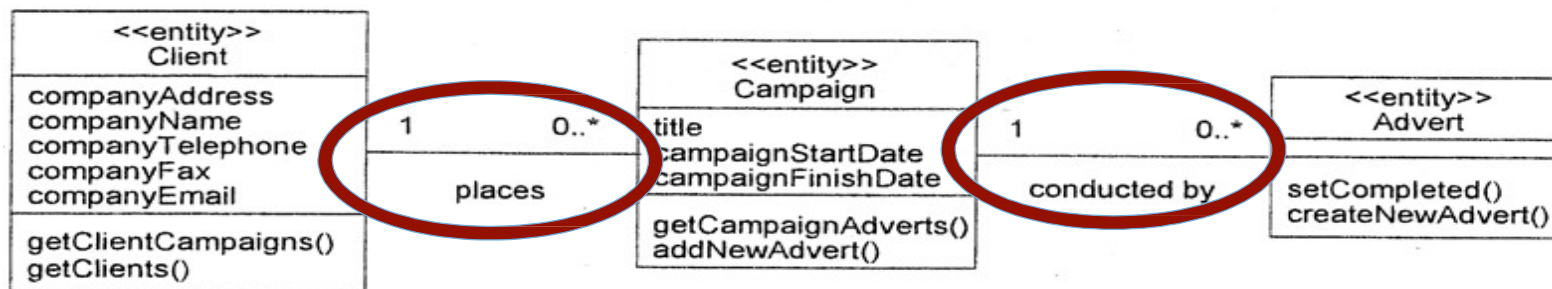
~colors : Color [3]

points : Point [2..*] {ordered}

name : String [0..1]

类关系

- 对象并非遗世独立，对象间存在千丝万缕的联系
 - UML中，关注以下几种类型的关系：
 - 关联关系 (Association)
 - 聚合与组合关系 (Aggregation and Composition)
 - 泛化关系 (Generalization)
 - 依赖关系 (Dependency)
 - 实现关系 (Realization)
- 类图描述类和它们之间的关系



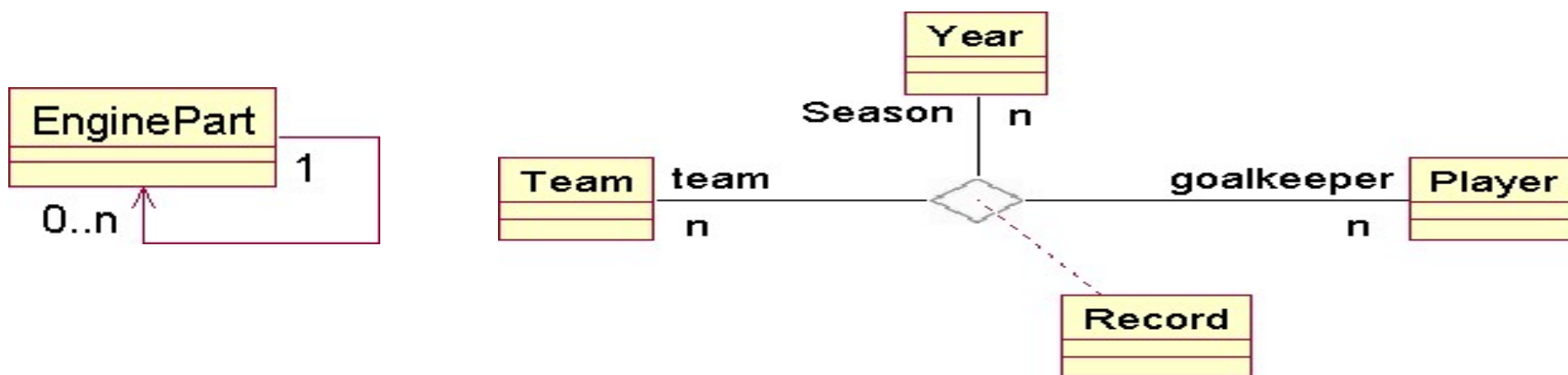
关联关系的“多样性/维度” (Multiplicity)

- 定义关联关系时尝试回答以下问题：
 - **市场活动 (Campaign) 类对象能否在没有员工 (StaffMember) 类对象管理的时候独立存在?**
 - 如果可以, 则员工 (StaffMember) 类的一端是可选的, 多样性/维度定义为 0..*
 - 如果不行, 则员工 (StaffMember) 类是必选的, 多样性/维度定义为 1..*
 - 如果必须有, 且只能由一个Staff来管理, 多样性/维度定义为 1
 - **然后再就另一端问同样的问题**
 - 每位员工都必须管理, 且只能管理一项市场活动吗?
 - 如果不是, 则市场活动一端的多样性/维度定义为0..*
- 其他的多样性定义举例:

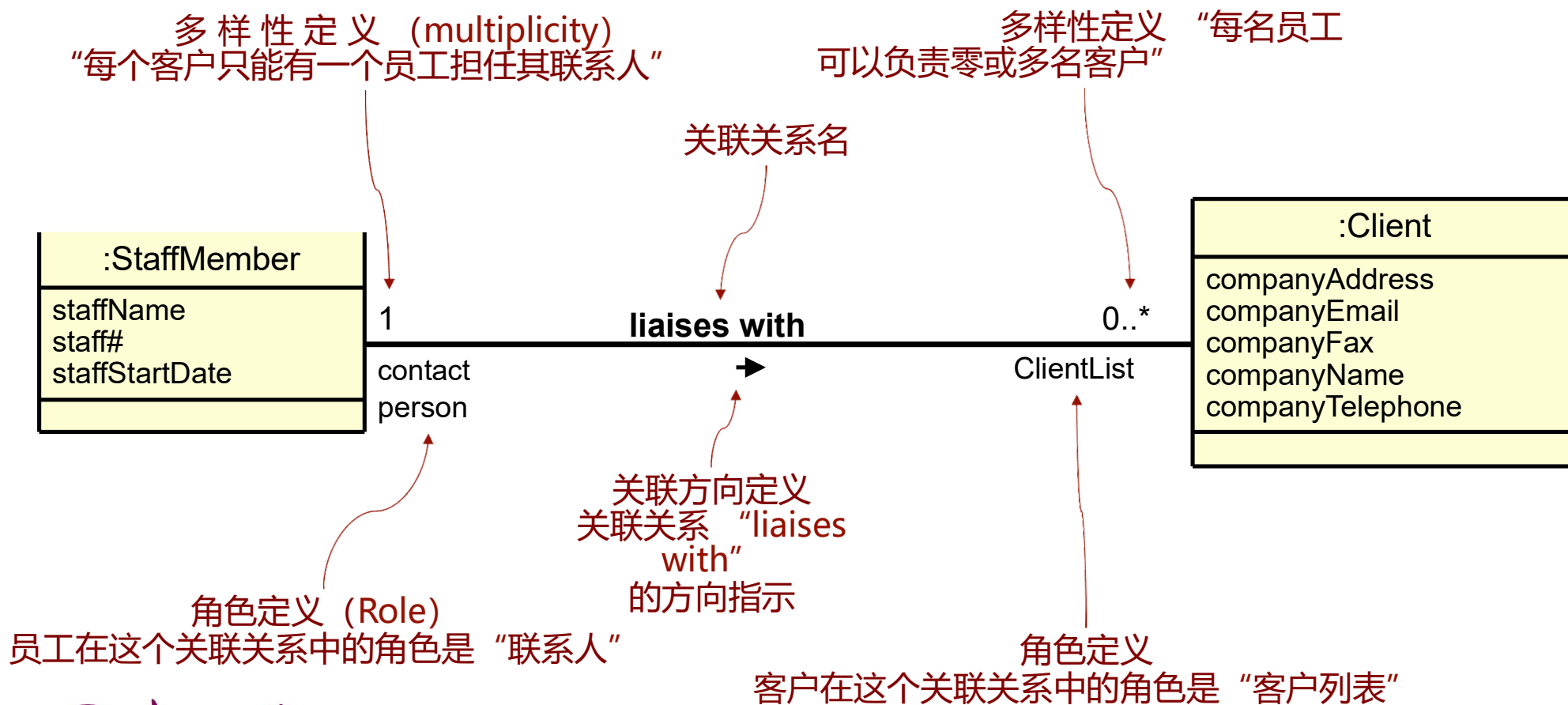
• 可选的 (Optional, 0 or 1)	0..1
• 有且仅有一个 (Exactly one, 1)	1..1
• 零或多个 (Zero or more)	0..*, *
• 一或多个 (One or more)	1..*
• 取值范围 (A range of values)	2..6

关联关系的种类

- 按照关联所连接的类的数量，对象类之间的关联可分为：**自返关联**，**二元关联**，**N元关联**
- 自返关联(reflexive association)**又称递归关联(recursive association)，是一个类与本身的关联，即同一个类的两个对象间的联系。
- 二元关联(binary association)**：二元关联是在两个类之间的关联。
- N元关联(n-ary association)**: 是在3个或3个以上类之间的关联。

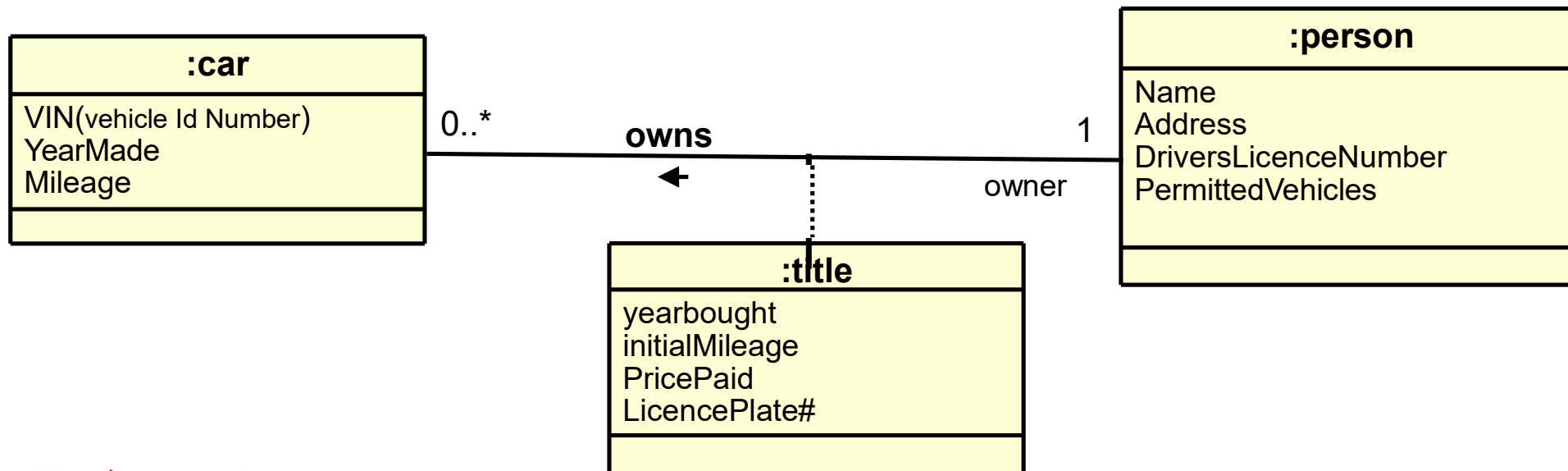


关联关系图例

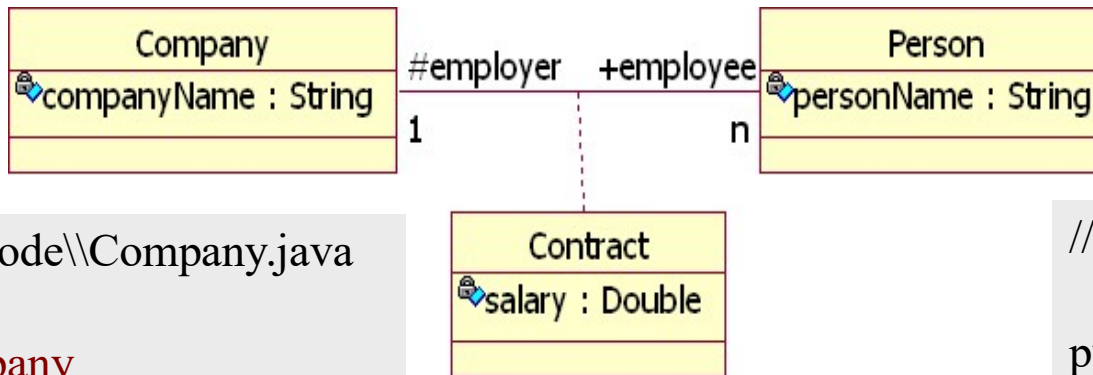


关联类

- 有时要为关联相关信息的存储定义一个专门的类，称为“关联类”
 - …保存与关联关系本身相关的信息
 - …这些信息不属于关联所连接的两端的类
 - 例如，下图中，“title”类的对象中存储的是车主和车辆之间所属关系有关的信息



关联类所生成Java代码示例：



//Source file: F:\\code\\Company.java

```
public class Company
{
    private String companyName;
    public Person employee[];
}
```

//Source file: F:\\code\\Contract.java

```
public class Contract
{
    private Double salary;
}
```

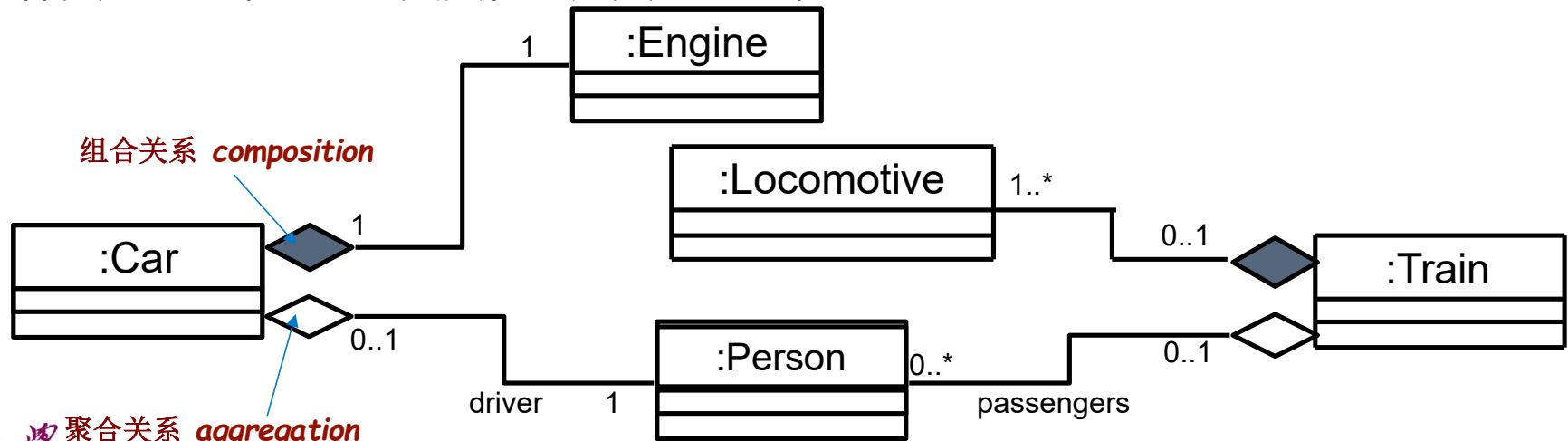
//Source file: F:\\code\\Person.java

```
public class Person
{
    private String personName;
    protected Company employer;
}
```



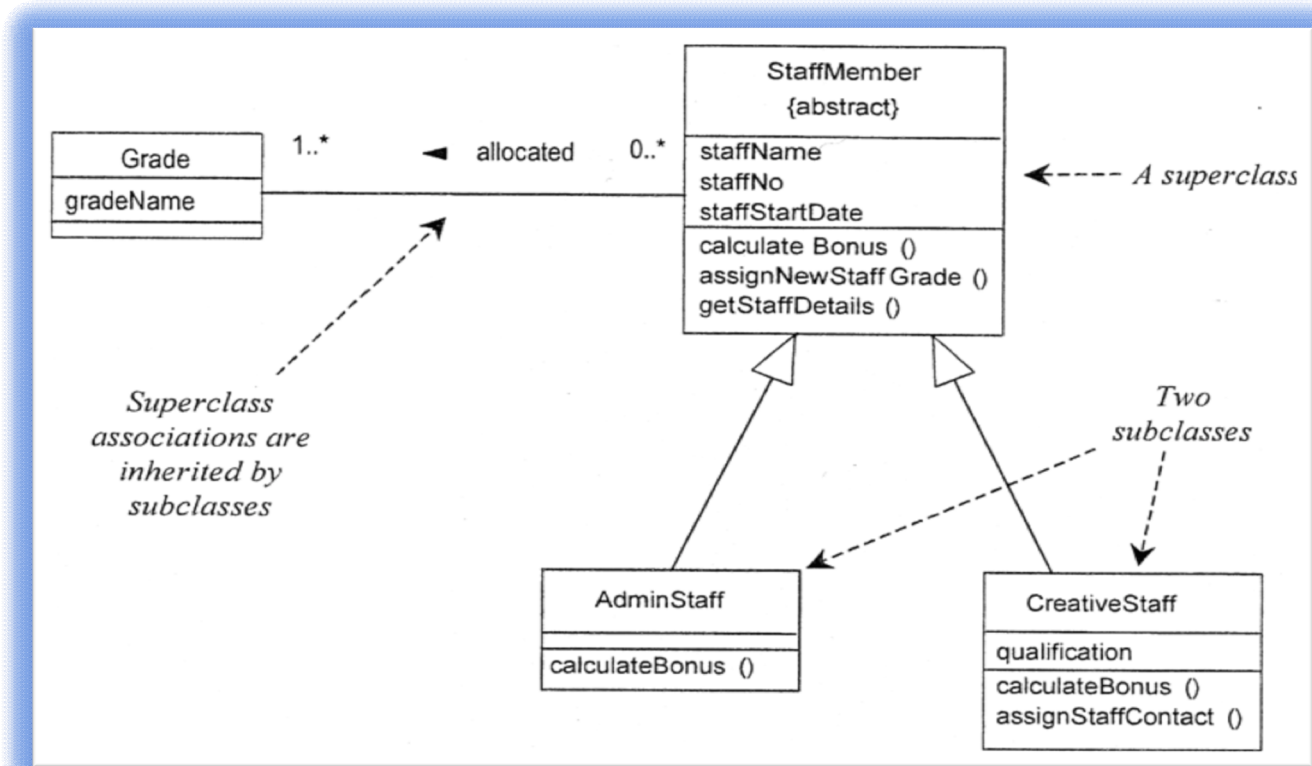
聚合 (Aggregation) 与组合 (Composition)

- 聚合 (Aggregation) 用于表达一个整体对象与其成员对象之间的关系
 - “**Has-a**” 或是 “**Whole/part**”
- 组合 (Composition) 用于表达一个整体对象与其组成部分之间的关系
 - 组合关系所表达的整体类与部分类之间的所属关系更强
 - 整体类的对象不存在时，部分类的对象也不存在
 - 整体类对象撤销之前要负责将部分类对象撤销



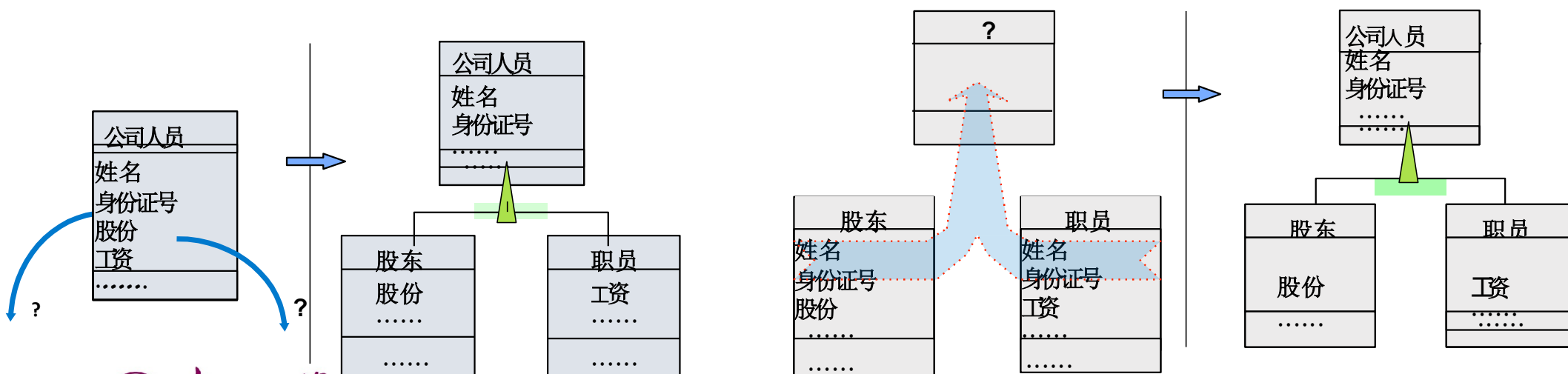
继承/泛化(Inheritance/Generalization)

- 子类继承父类的属性、关联和操作
- 子类可以覆盖继承来的内容
 - 例如：行政员工AdminStaff和专业设计人员CreativeStaff的奖金计算方法可以不同
- 父类可以声明为抽象类{abstract}，则
将不会为它直接创建实例对象
 - 意味着现有子类覆盖了该类对象的全集
- 例如，除了行政员工AdminStaff和专业设计人员CreativeStaff之外，
不存在其他类员工

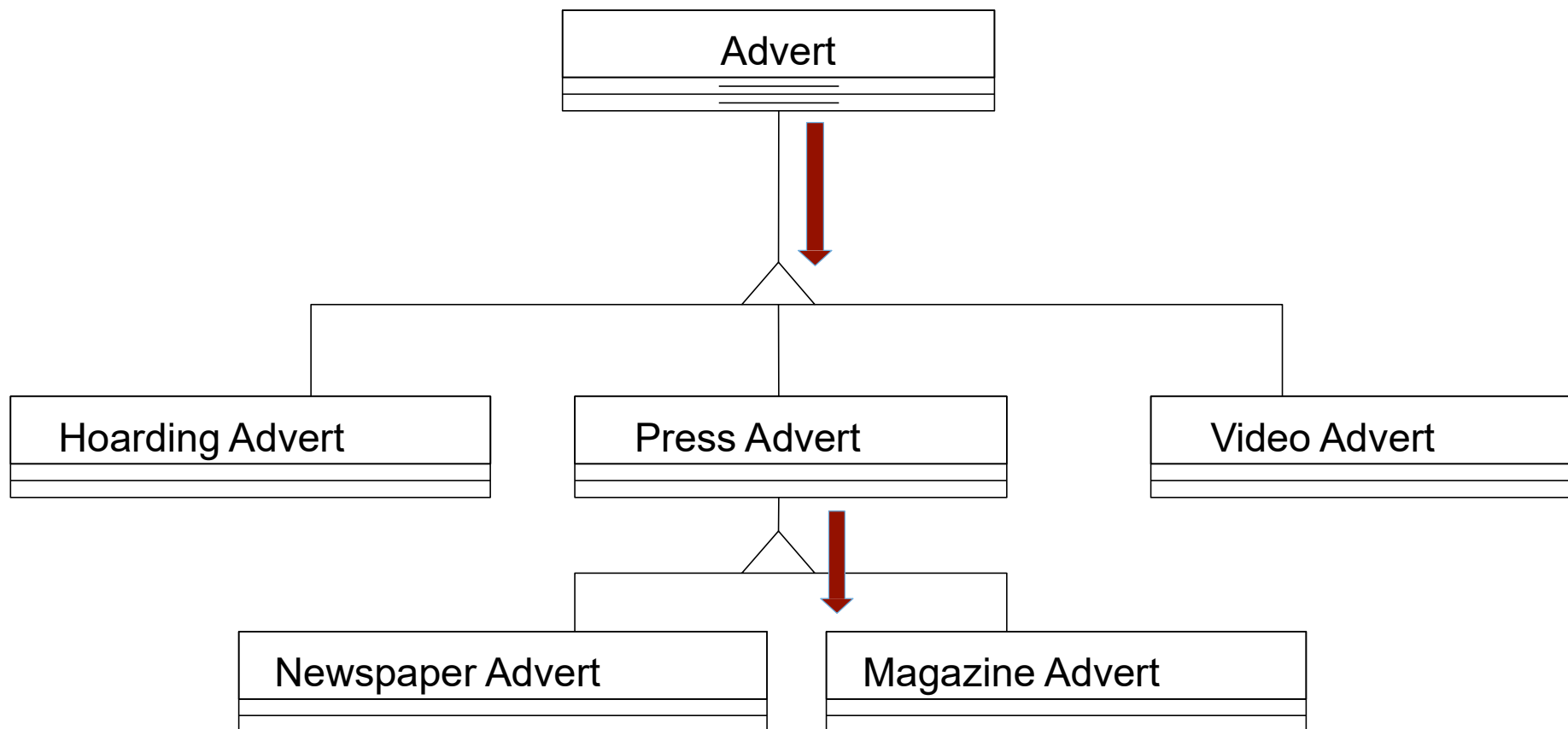


继承/泛化(Inheritance/Generalization)关系的定义

- 继承/泛化关系建模的意义在于系统环境发生变化时便于添加新的子类
- 继承/泛化关系建模的过程
 - **自顶向下**
 - 将某个类分割为属性和操作不同的子类，或者发现关联关系定义的是分类关系 “kind of”
 - **自底向上**
 - 为现有的多个具有公共属性及方法的类，定义一个父类

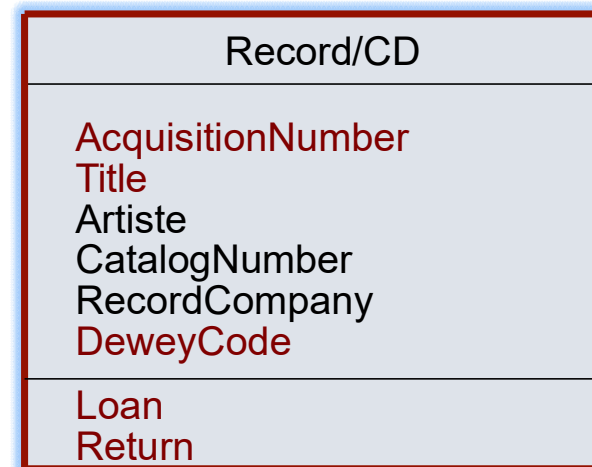
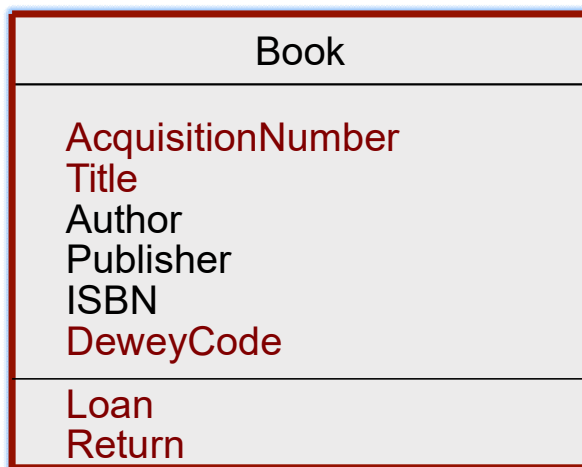


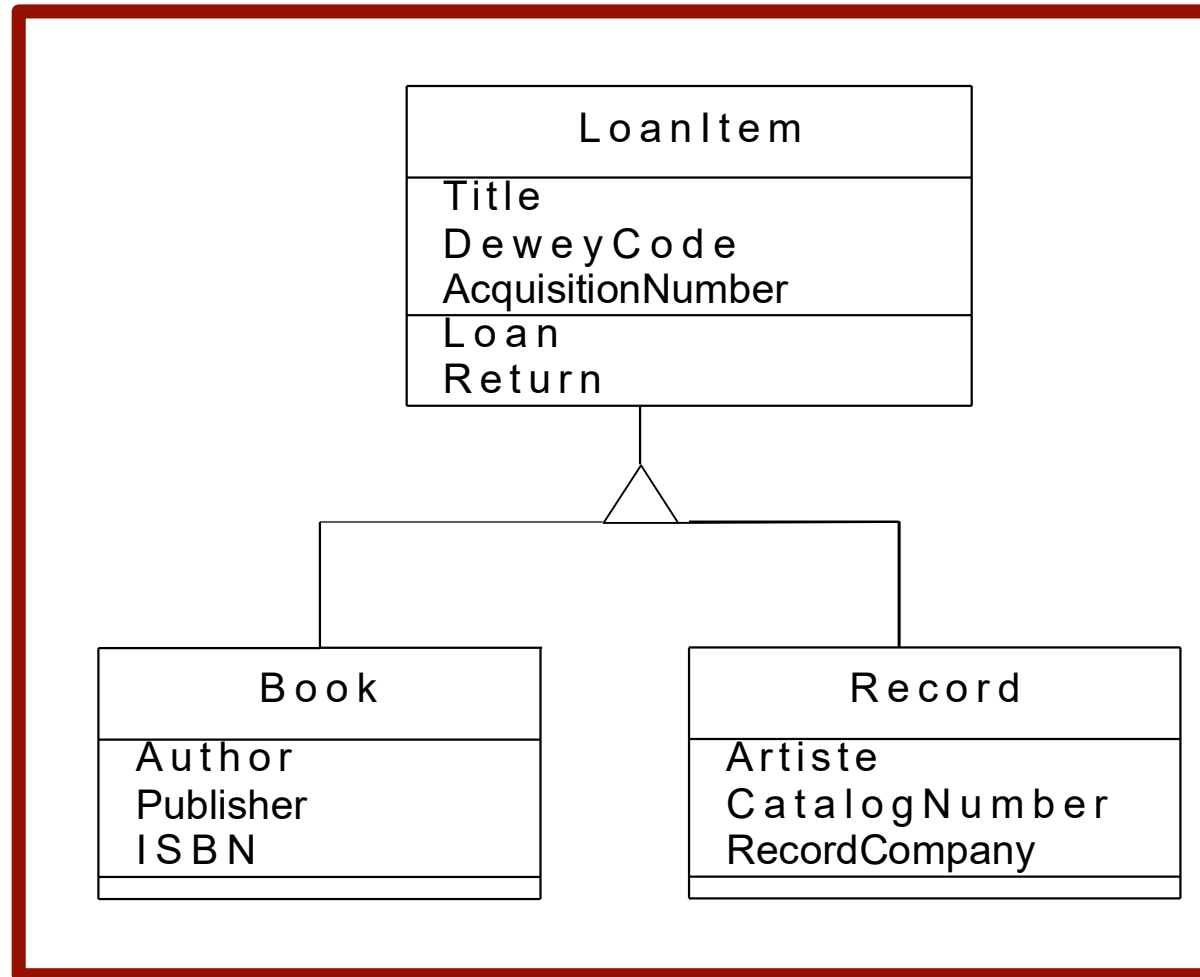
自顶向下定义继承关系



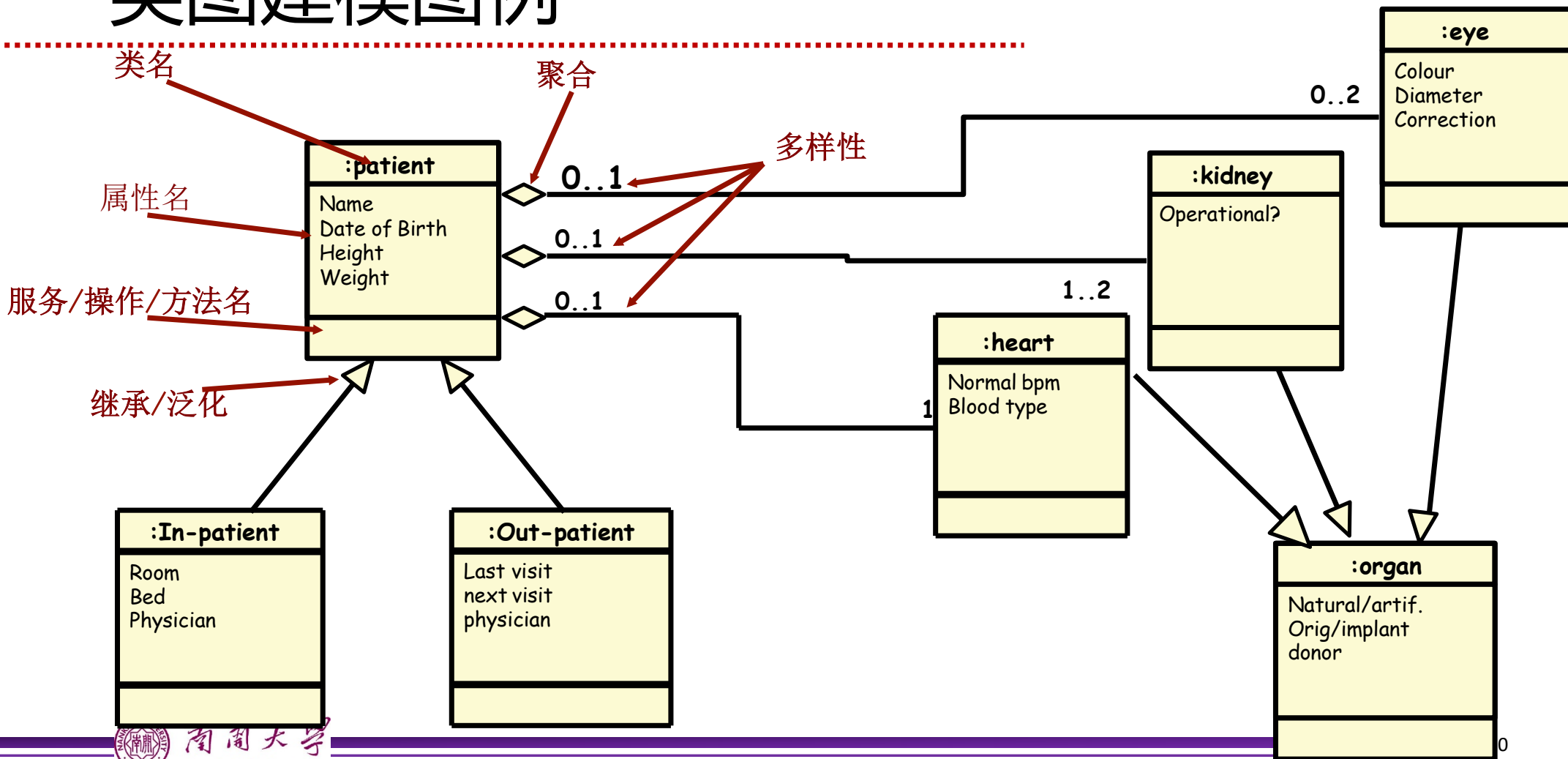
自底向上定义继承关系

为右图的两个类定义公共父类，
并重画该类图



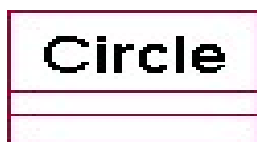


类图建模图例



类图的抽象层次

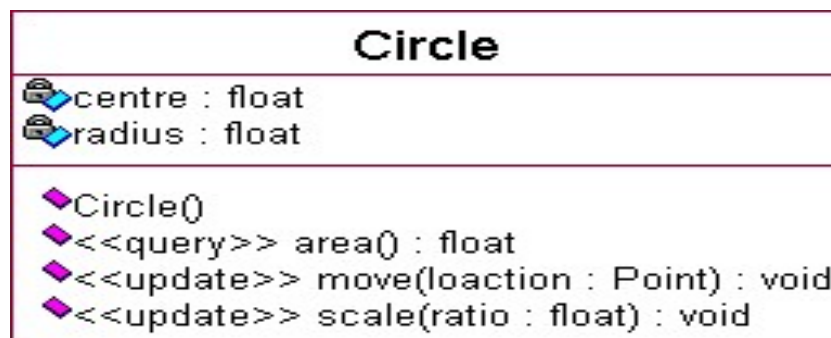
- 在软件开发的不同阶段使用的类图具有不同的抽象层次。一般地，类图可分为三个层次，即 **概念类**，**设计说明类**和**实现类**。
- 概念类**，**设计说明类**和**实现类**的划分最先是由 **Steve Cook** 和 **John Daniels** 引入的。例：**类图的三个层次**的例子。



概念类



设计说明类



实现类

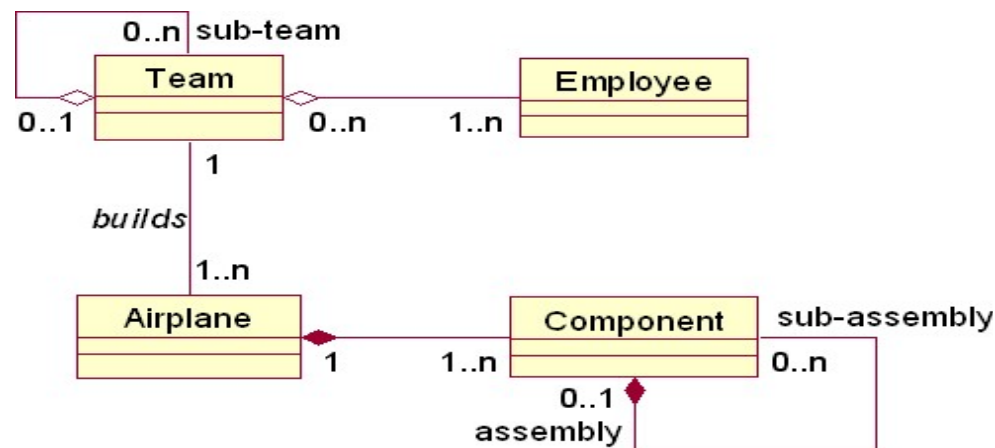
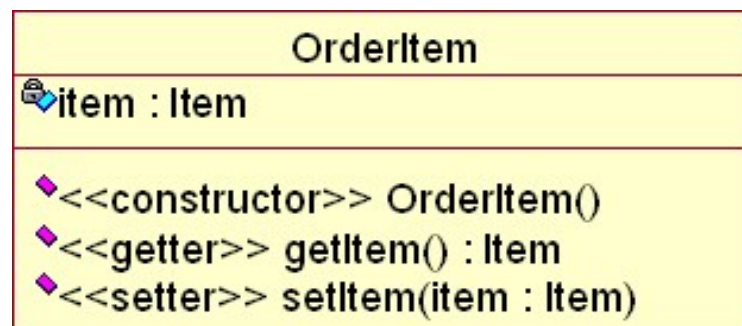


总结建立类图的步骤

- 1.研究分析问题领域，确定系统的需求。
- 2.发现对象与类，明确它们的含义和责任，确定属性和操作。
- 3.发现类之间的关系。把类之间的关系用关联、泛化、聚集、组合、依赖等关系表达出来。
- 4.设计类与关系。调整和细化已得到的类和类之间的关系，解决诸如命名冲突、功能重复等问题。
- 5.绘制类图并编制相应的说明。

类图建模风格

- 1: 属性名和类型应该一致
- 2: 不要对有关联类的关联关系进行命名。
- 3: 静态操作/属性要在实例操作/属性之前列出。
- 4: 以可见性降低的次序列出操作/属性。
- 5: 避免已被语言的命名规范所隐含的版型。
- 6: 总是指明多样性
- 7: 不要对每个依赖关系都建模
- 8: 将子类放在父类的下方
- 9: 小心基于数据的继承
- 10: 按惯例是把整体画在部分的左边



生物学的分类体系中把企鹅归属为鸟类。我们模仿这个体系，让企鹅继承鸟这个类，而类“鸟”中有个方法fly，企鹅自然也继承了这个方法，可是企鹅不能飞，于是，我们在企鹅的类中覆盖了fly方法，告诉方法的调用者：企鹅是不会飞的。

请问上述做法违反了以下哪个原则？

提交

- A 依赖倒置原则
- B 开闭原则
- C 里氏替换原则
- D 接口分离原则

关于UML类图中泛化关系建模，不正确的说法是？

提交

- A 定义泛化关系的好处是当环境发生变化时，便于添加新的子类
- B 当某个类中，存在属性和操作略有不同的子类时，应定义泛化关系
- C 当现有的多个类具有公共属性和方法时，可以定义一个父类让它们共同继承
- D 关联关系也可以用于定义分类关系，可以替代泛化关系





第4.3节 交互行为建模

Modelling Interactions

李起成

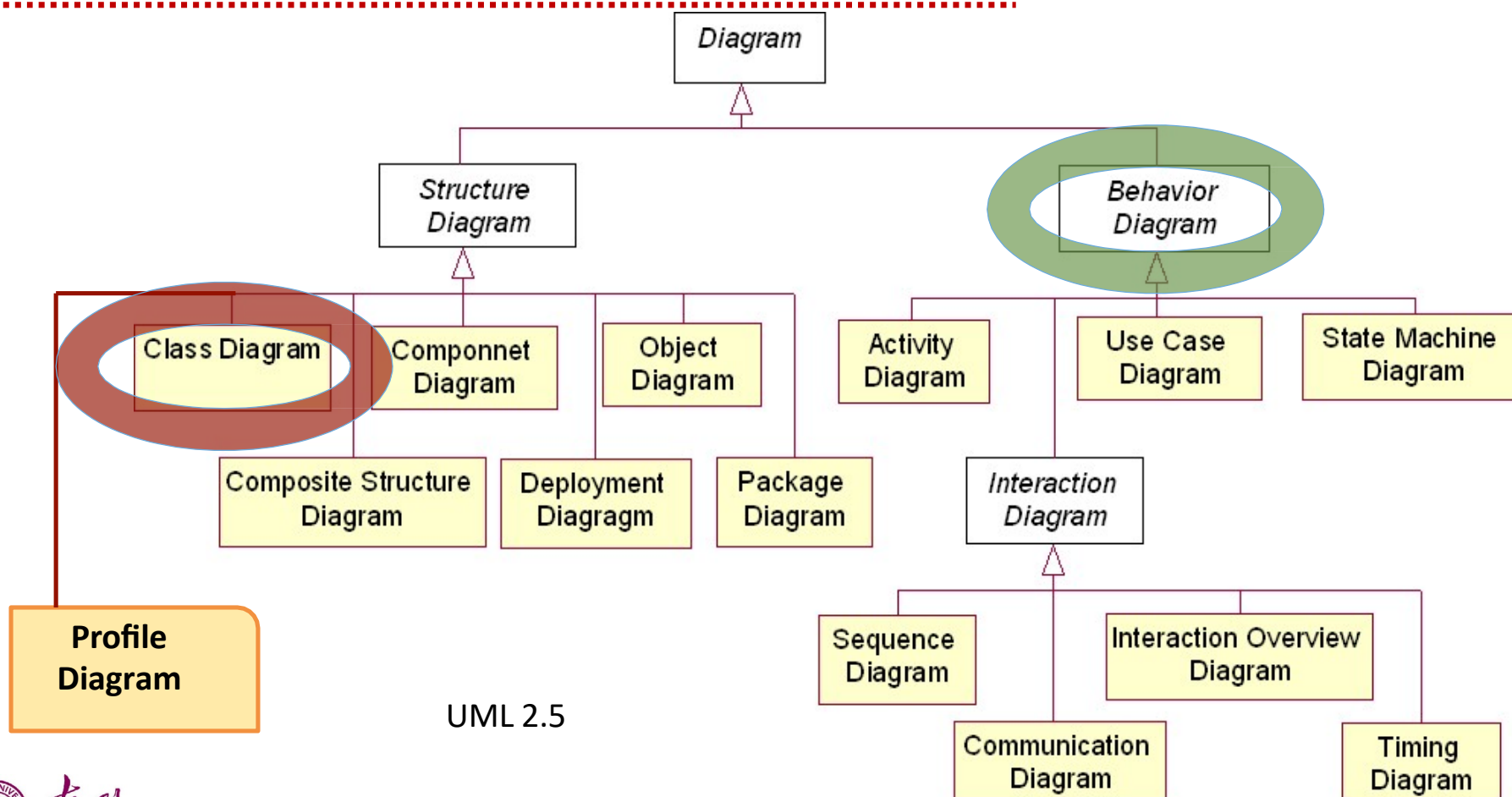
College of Computer Science
Nankai University

*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南開大學
Nankai University

UML 2.0中图的分类





顺序图

Sequence Diagram

李起成

College of Computer Science
Nankai University

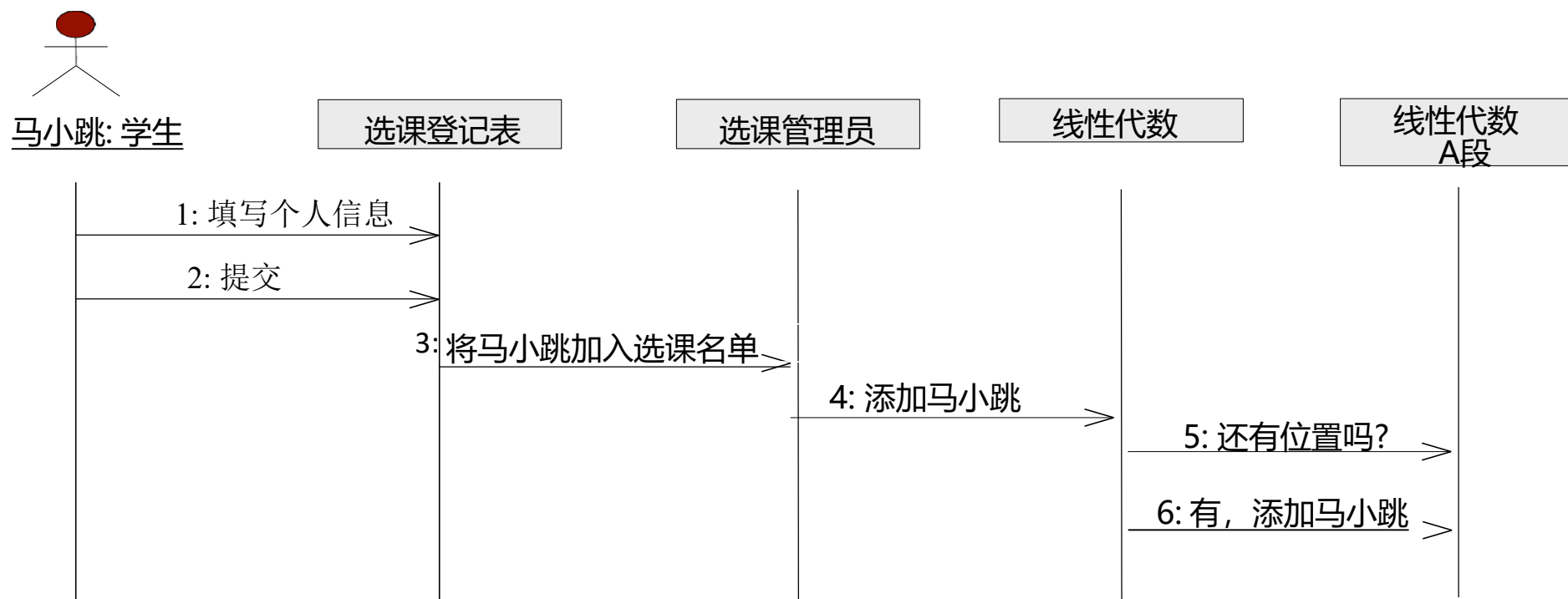
*Slides adapted from materials by Prof. Qiang Liu (Tsing Hua University) and
Ivan Marsic (Rugers University)*



南开大学
Nankai University

顺序图举例 (Sequence Diagram)

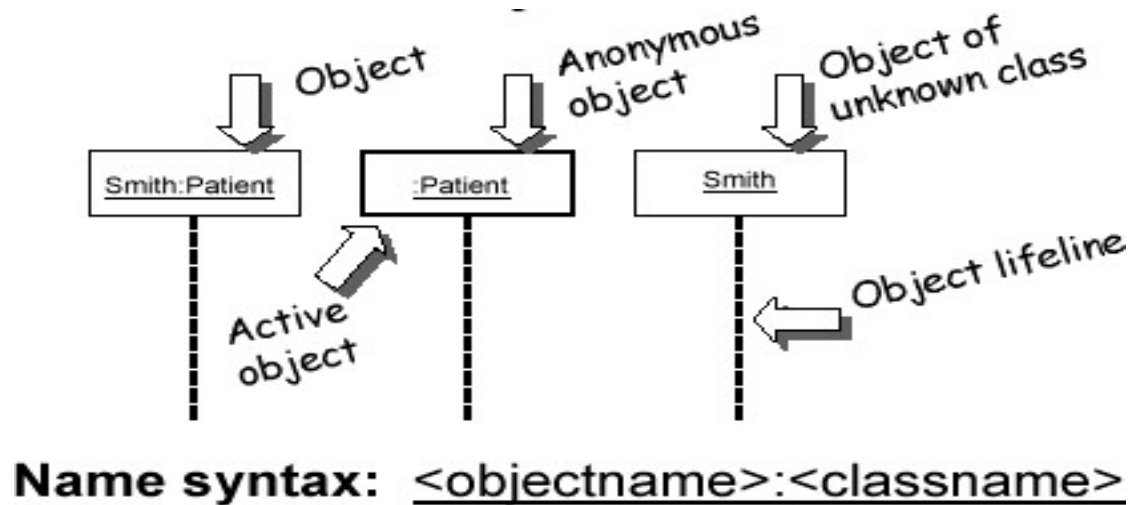
- 顺序图用来刻画系统实现某个功能的必要步骤



顺序图建模元素对象(Object)及其生命线(Lifeline)

对象：以某种**角色**参与交互，可以是人，物，其他系统或者子系统

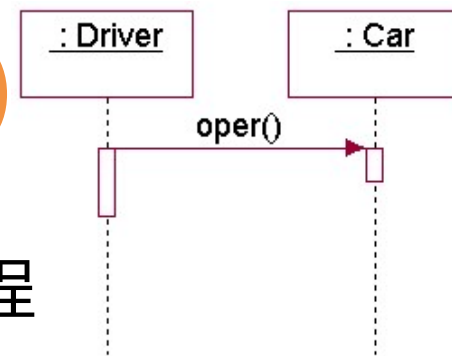
生命线：表示对象存在的时间



控制焦点/激活期(Focus of Control/Activation)：表示对象进行操作的时间片段



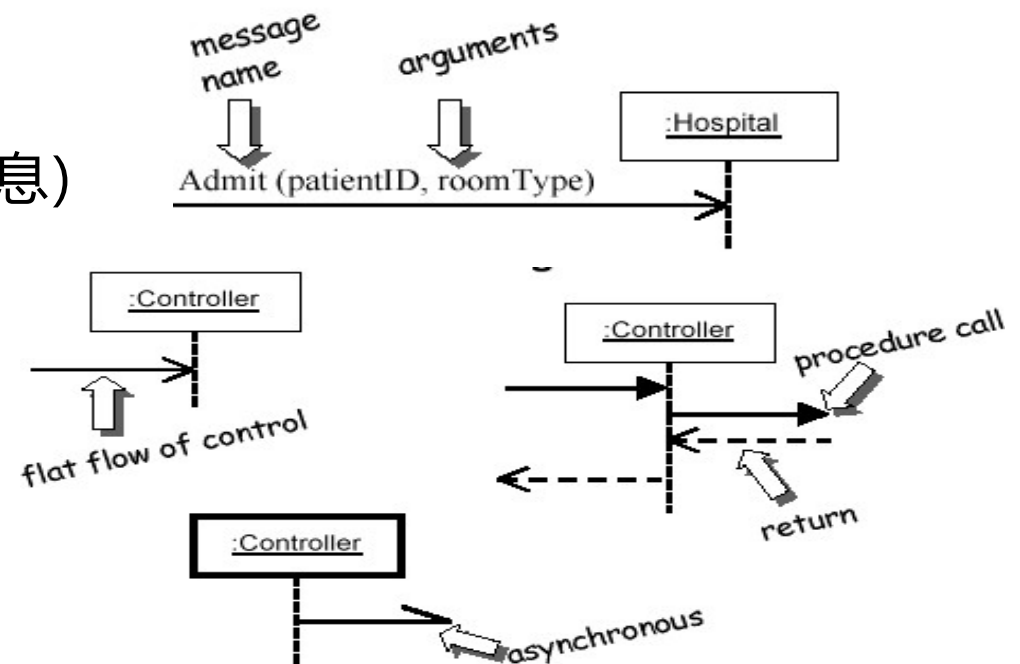
顺序图建模元素——消息(Message)



- 消息(Message)用于描述对象间的交互操作和值传递过程

- 消息类型:

- Synchronous 同步消息 (调用消息)
- Return 返回消息
- Asynchronous 异步消息
- Self-message 自关联消息
- Time-out 超时等待
- Uncommitted / Balking 阻塞

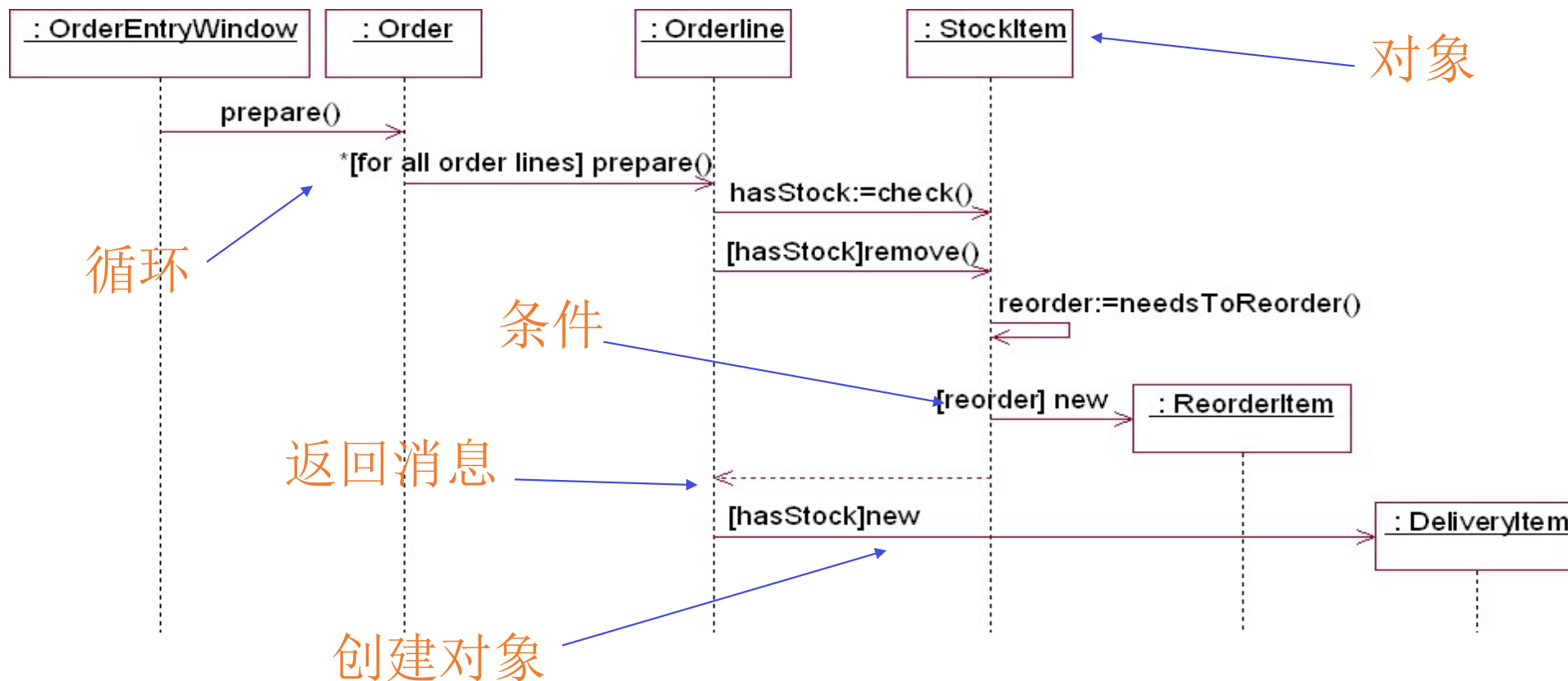


消息的表示形式

例：一些消息的例子

2: display (x, y)	简单消息
1.3.1: p:= find(specs)	嵌套消息，消息带返回值
4 [x < 0] : invert (x, color)	条件消息
3.1*: update ()	循环消息

顺序图中的基本结构



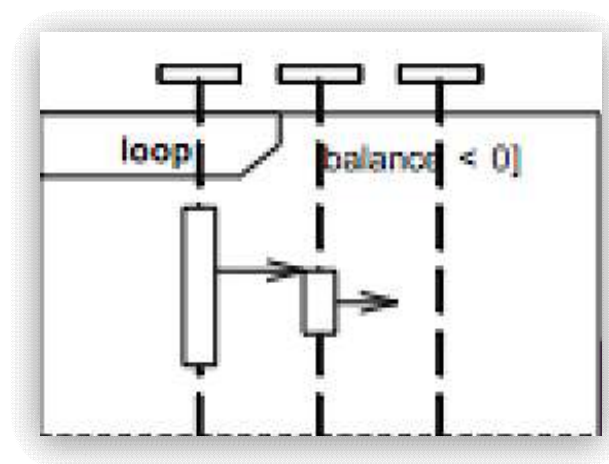
顺序图中消息的循环发送

- 在消息名字前加循环条件或添加循环控制框

例：

1.1 *[for all order lines]: message1()

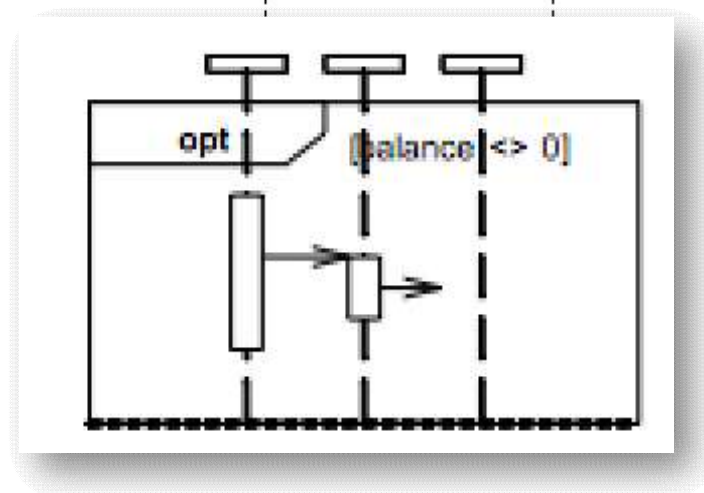
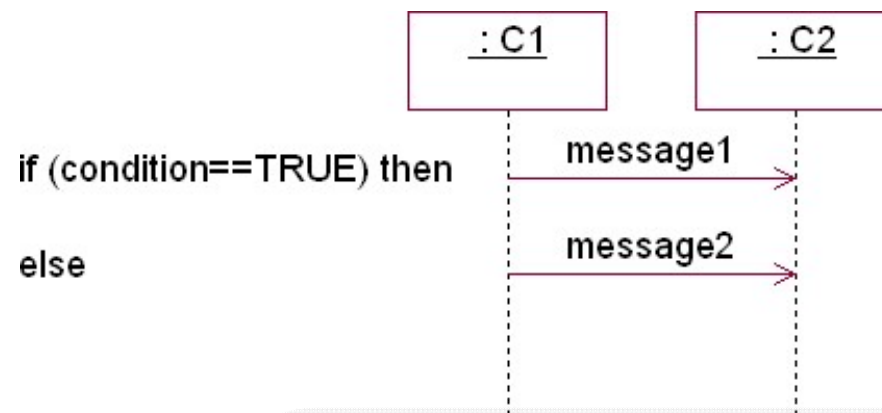
2.1 *[i:=1..n]: message2()



顺序图中带条件消息的发送

- 在消息名字前加条件子句;
- 使用文字说明;
- 添加条件控制框;
- 分成多个顺序图子图并关联

[hasStock]: remove()



绘制顺序图

1. 在顺序图顶端绘制矩形框，定义参与交互的**类实例**（对象）名；
2. 在每个对象下面绘制竖直虚线，表示该对象的**生命线**；
3. 在对象间添加箭头表示各种类型的**消息**，跟踪对象间的控制流；
4. 生命线加竖直矩形定义对象**激活期**，表明对象正在执行某操作；
5. 根据需要添加**框**的组合与关联，表示复杂的控制结构。

顺序图建模过程

• 参与者

顺序图中有关对象或者实体

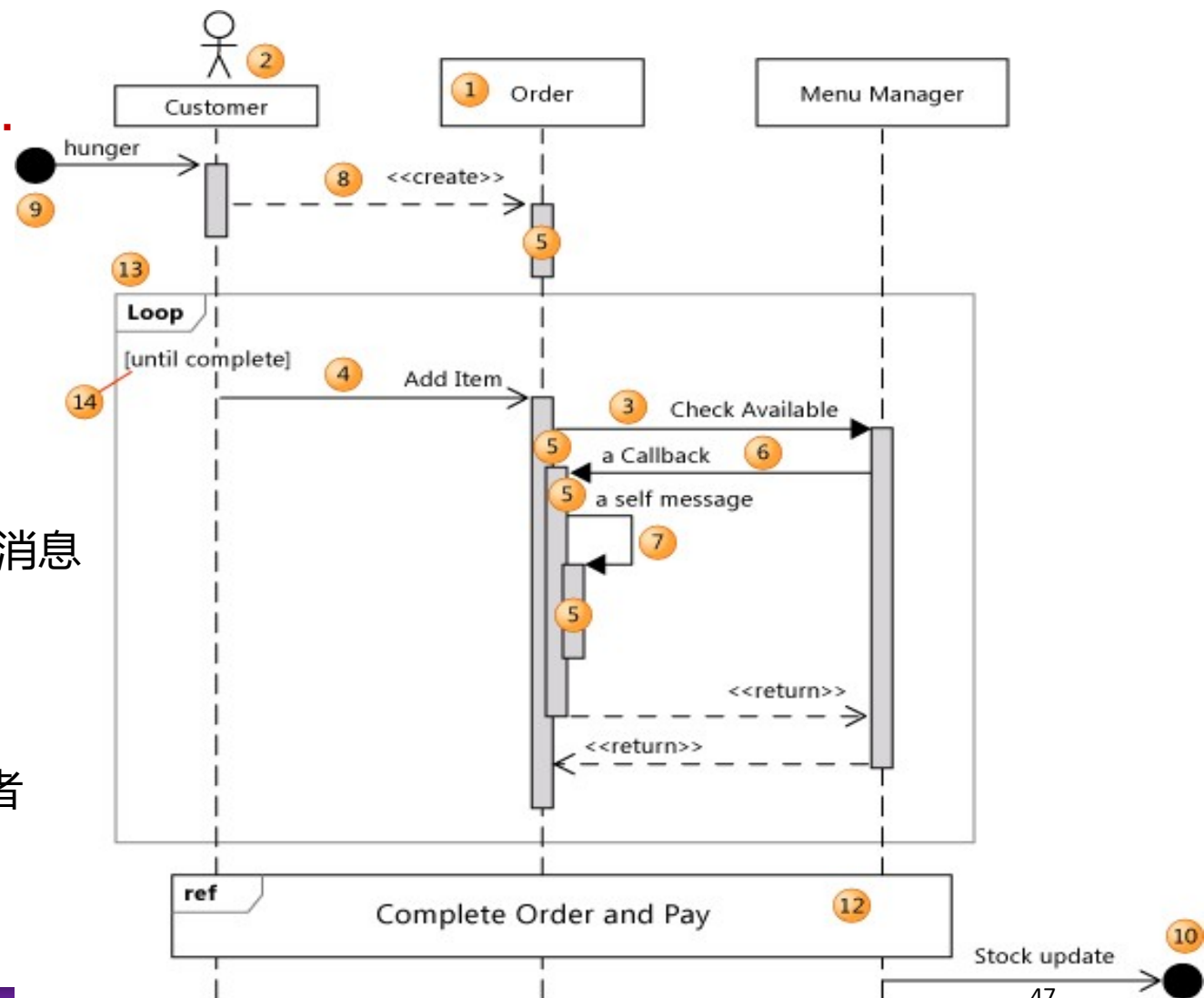
● 消息

参与对象之间的通信，通过箭头表示

- 顺序图的起始是一个没有发起对象的消息
- 每个消息代表的操作属于消息接受方

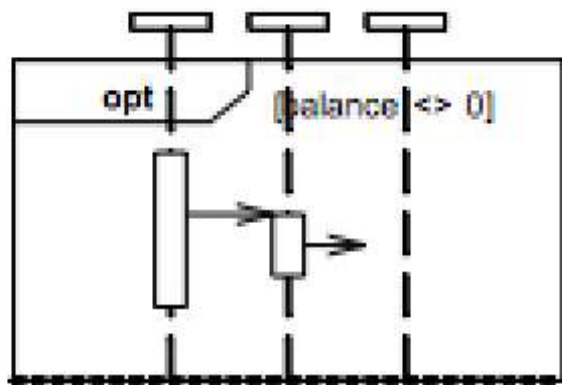
• 轴

- 横轴: 表明正在进行操作的对象/参与者
- 纵轴: 时间 (向下表明时间的顺延)

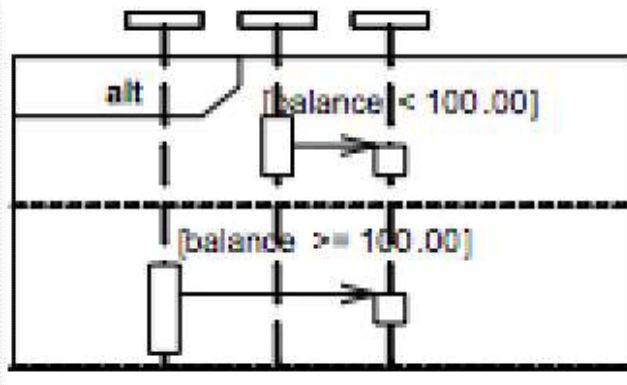


组合框：复杂控制结构表示

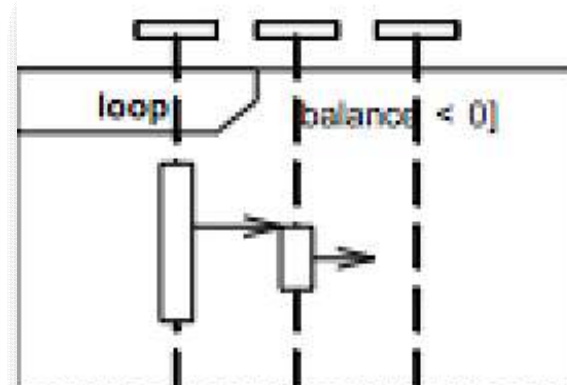
- **框 (Frame)**：框中包含顺序图的部分结构，表示选择(selection)或者循环(loop)结构，左上角注明结构类型，[]中注明条件。



if ->
(**opt**)[condition]



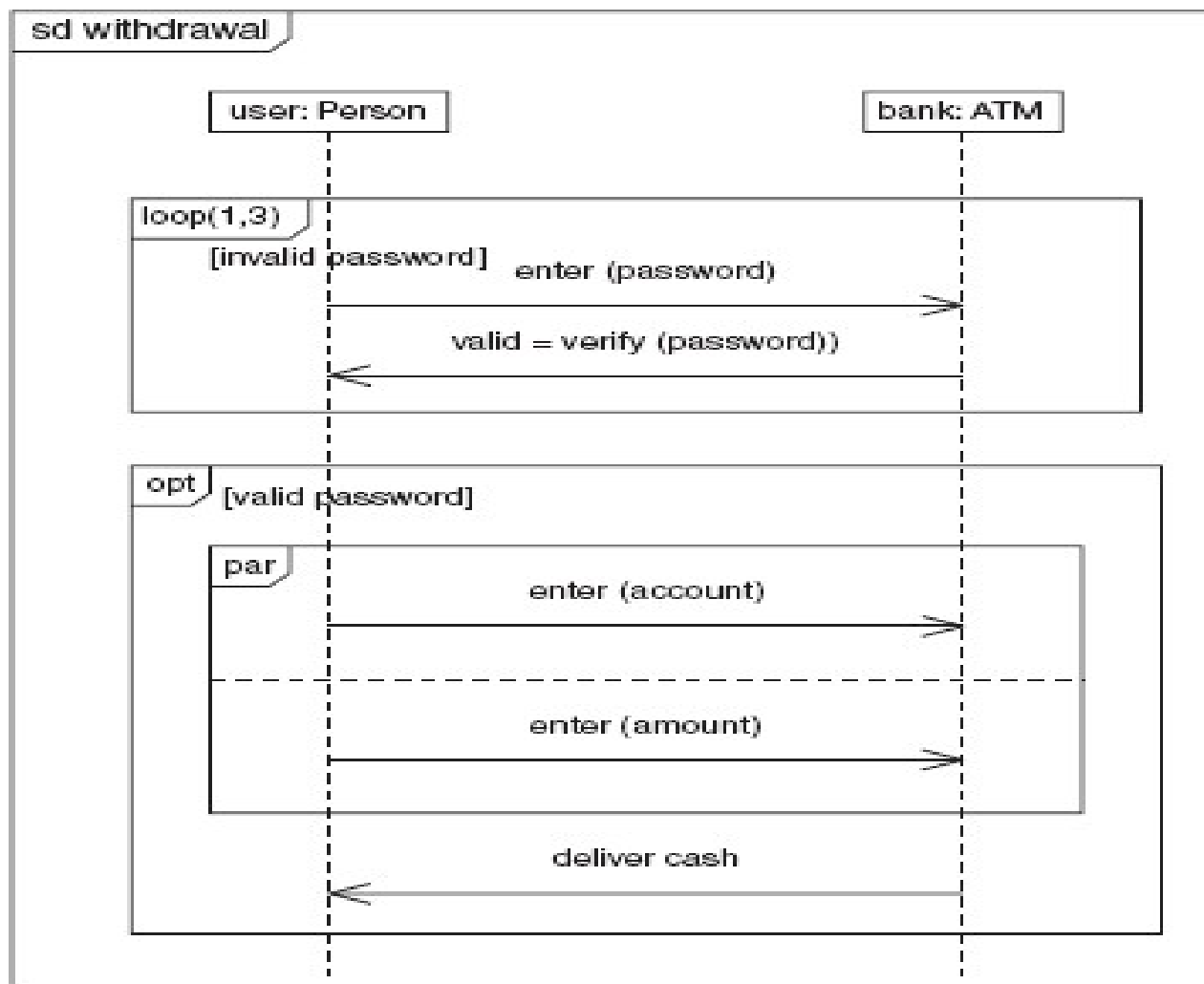
i f / e l s e ->
(**alt**)[condition] 通过水
平虚线分割不同情形 并发
结构 -> (**par**)



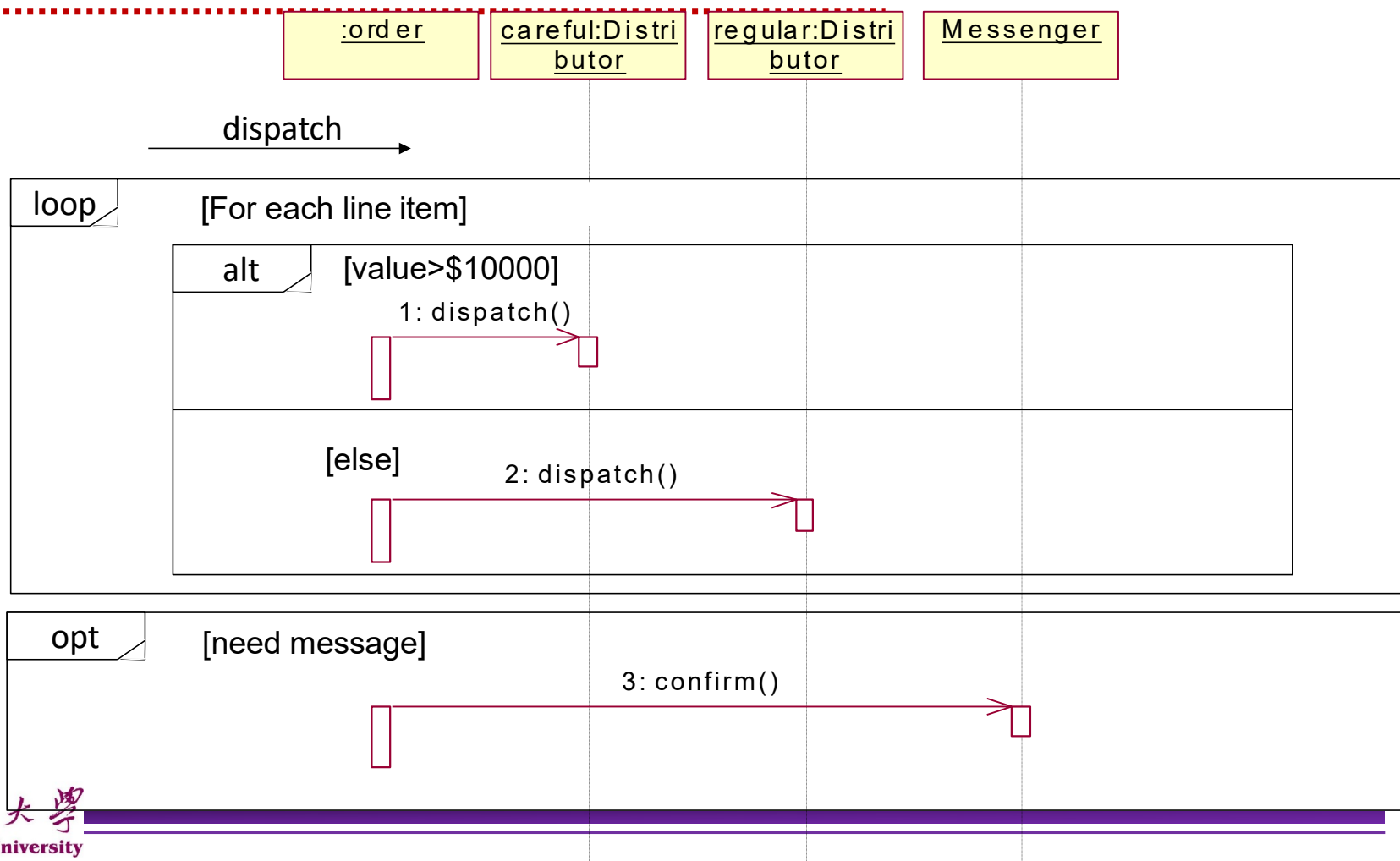
loop -> (**loop**)[condition or items to loop
over]



控制框建模例子

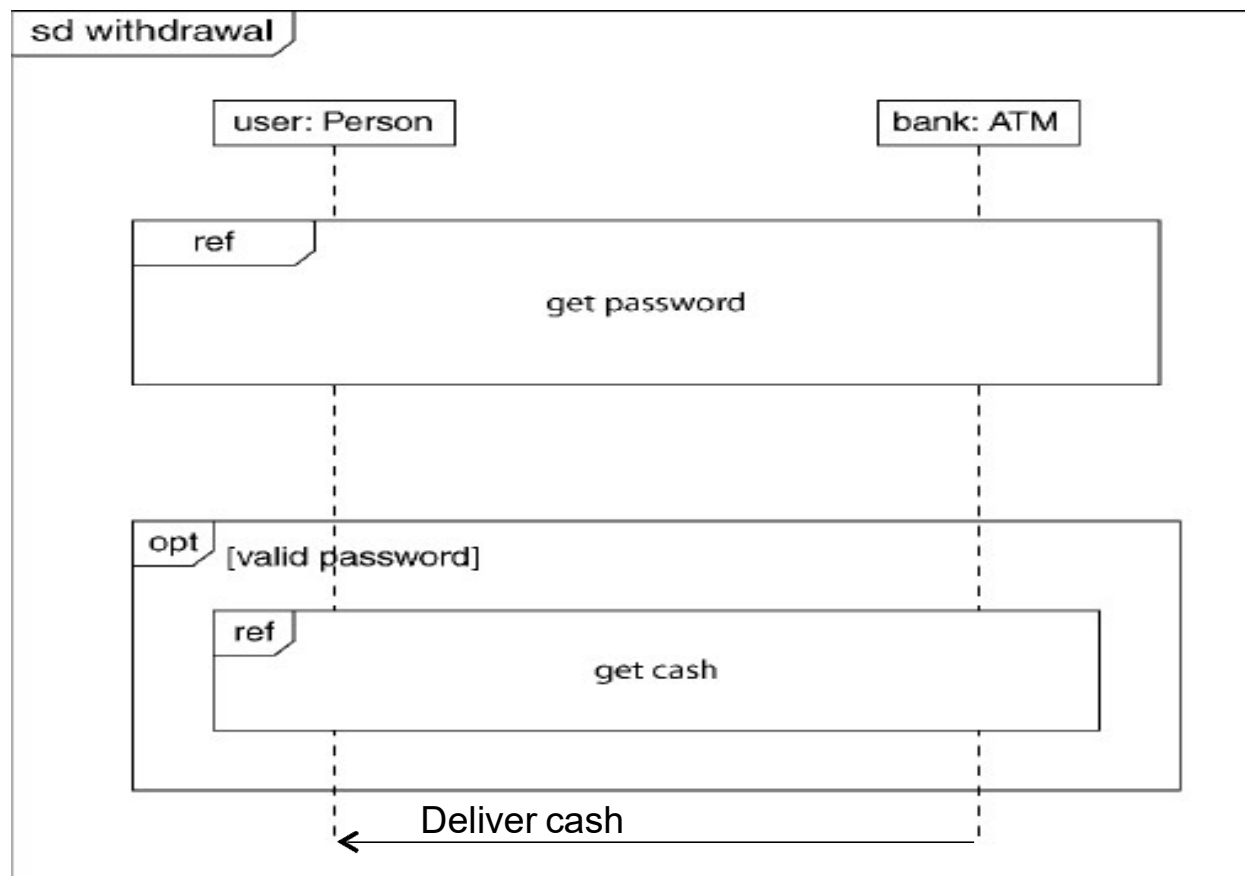


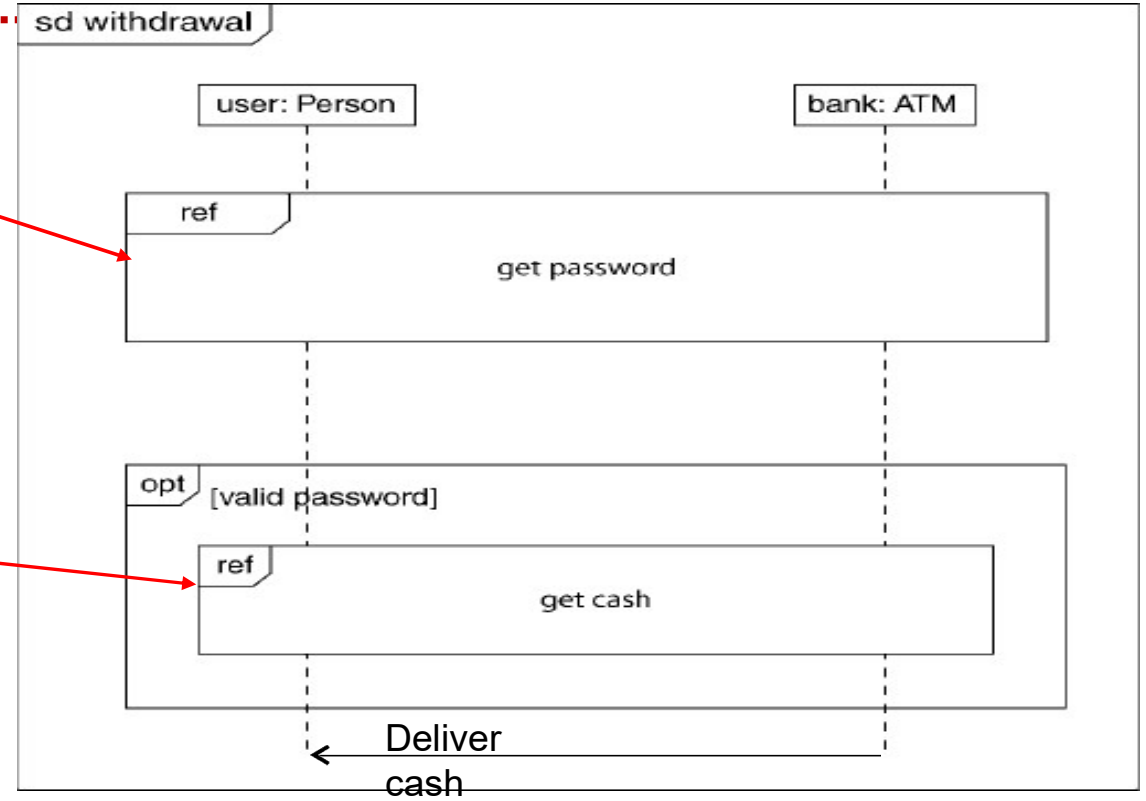
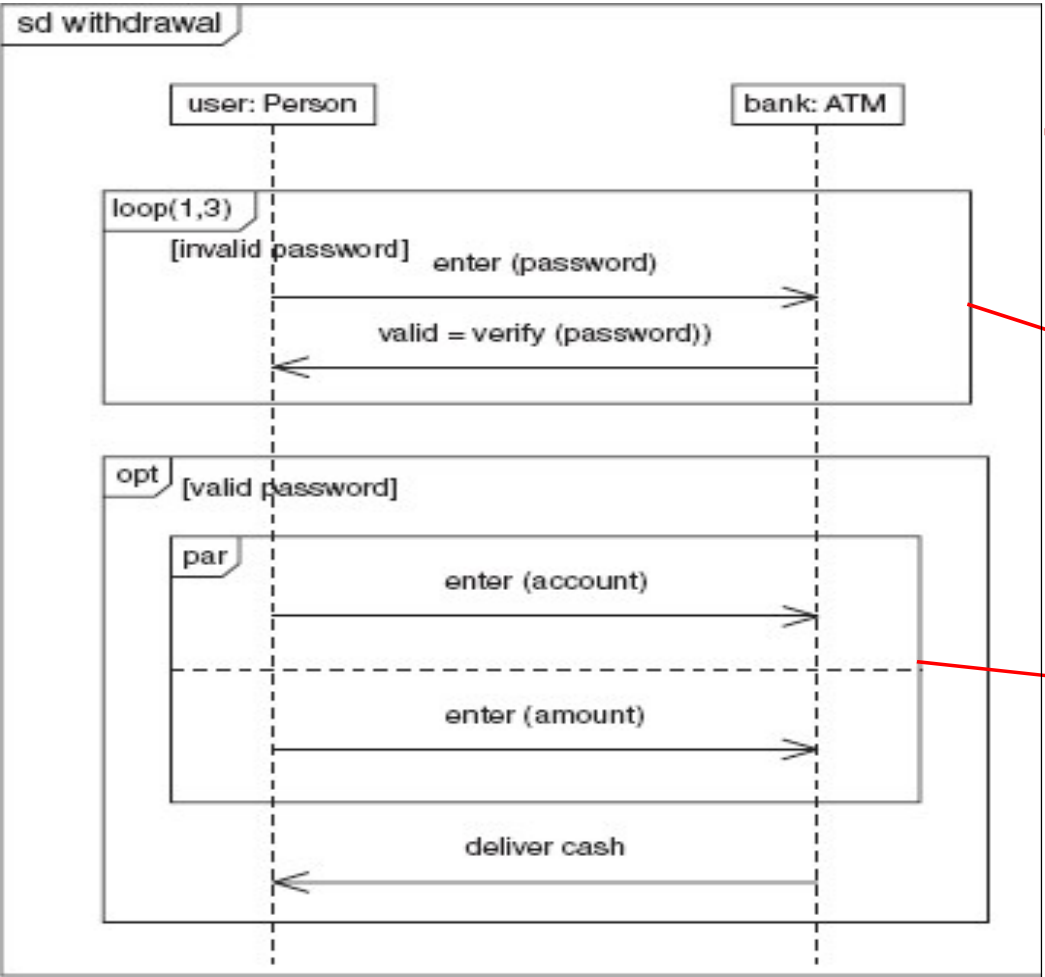
控制框建模例子



顺序图间的关联

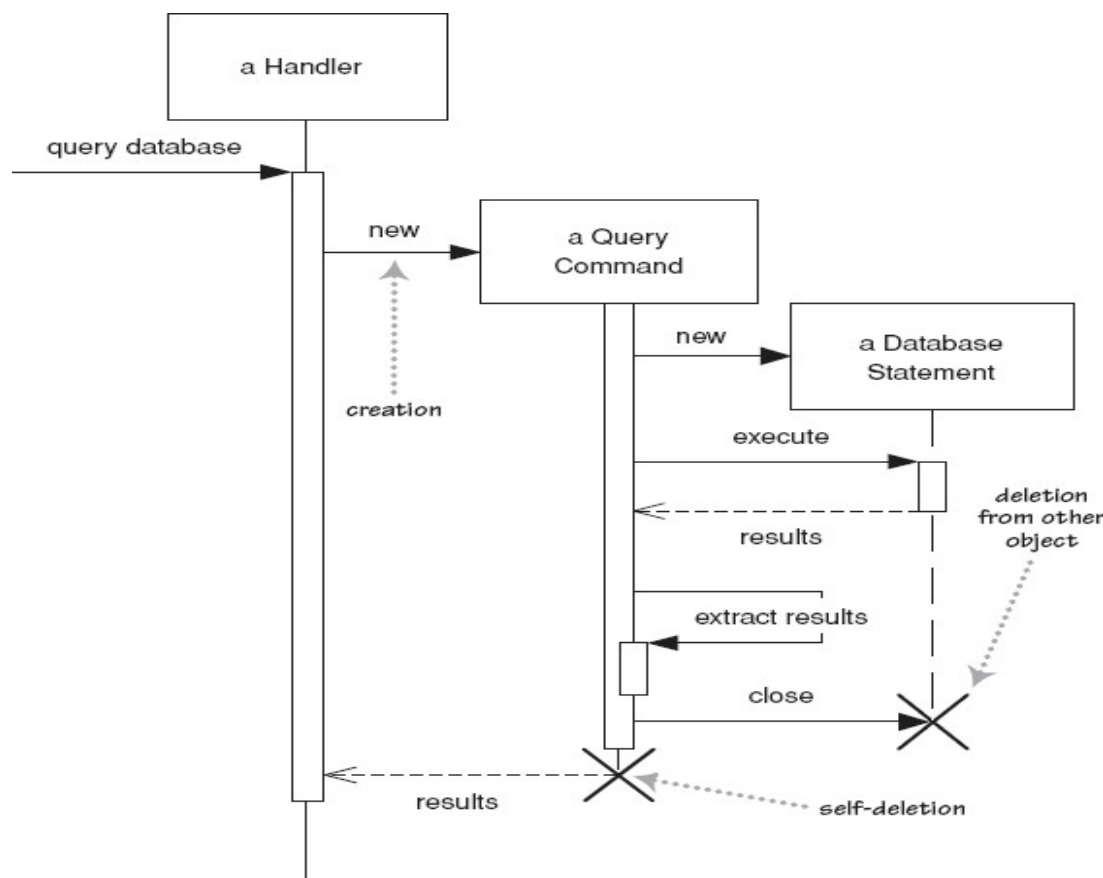
- 当一个顺序图过大
- 需要引用其他图表时,
选择下述表示:
 - 不完整的箭头和注释
 - 通过名为“ref”的
框图引用相关图表



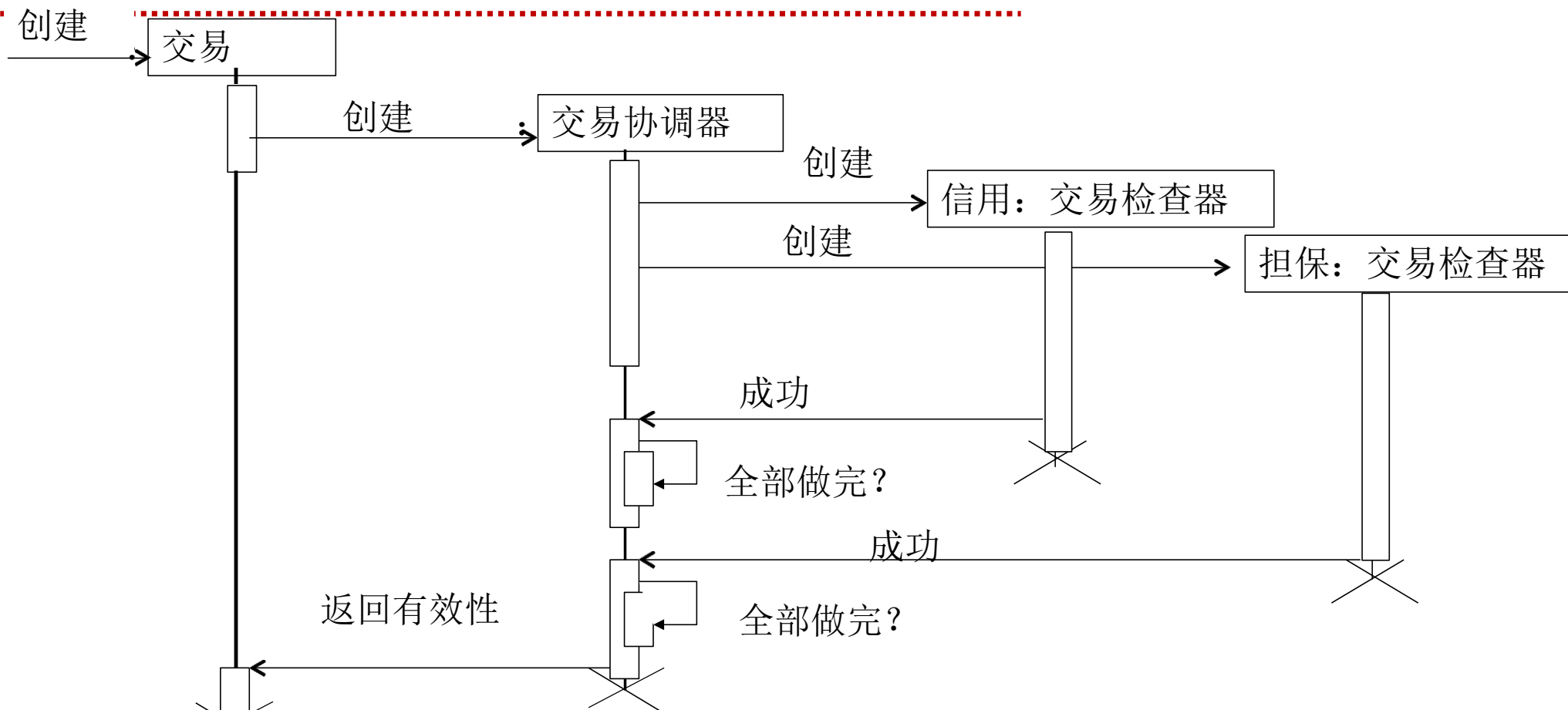


对象的创建与撤销

- 创建(creation): "new" 标明的箭头
 - 用例场景中新建的对象在图中的位置较低
- 撤销(deletion,destroy): 生命线底部的 "X"
 - 注: 在Java没有明确的销毁对象的操作, 通过垃圾回收机制处理

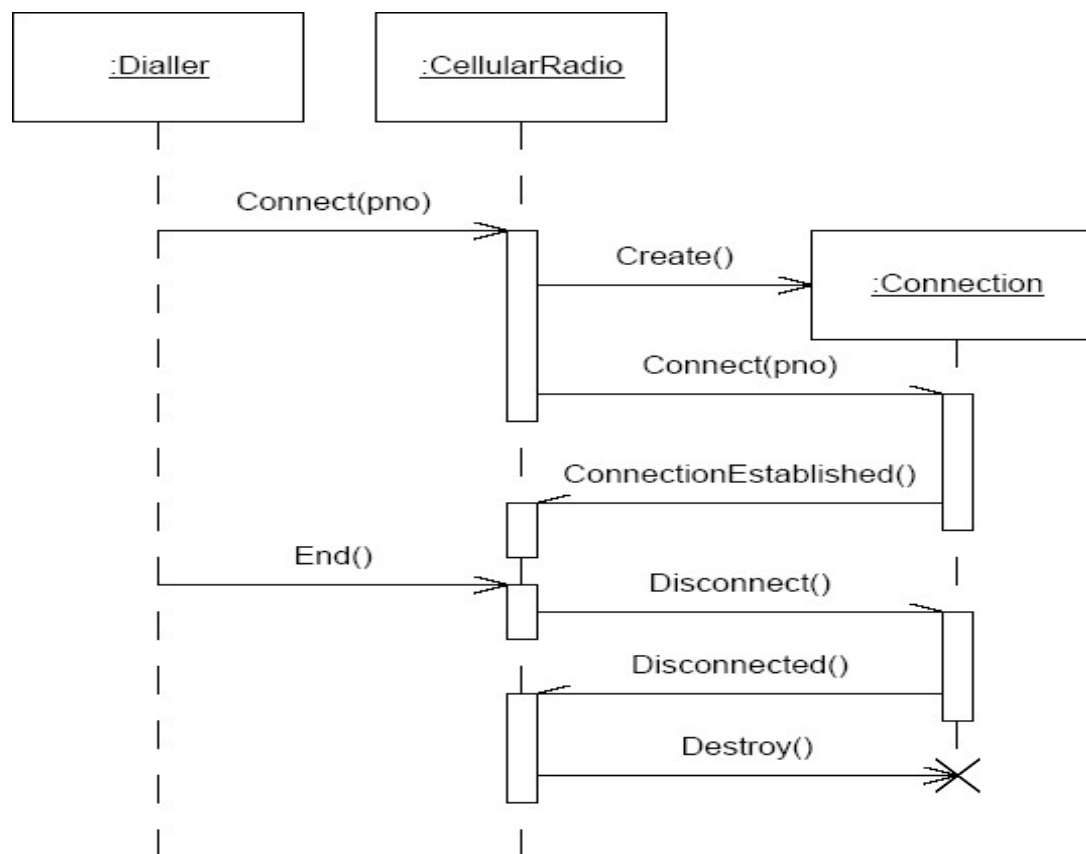


例:银行系统的交易验证

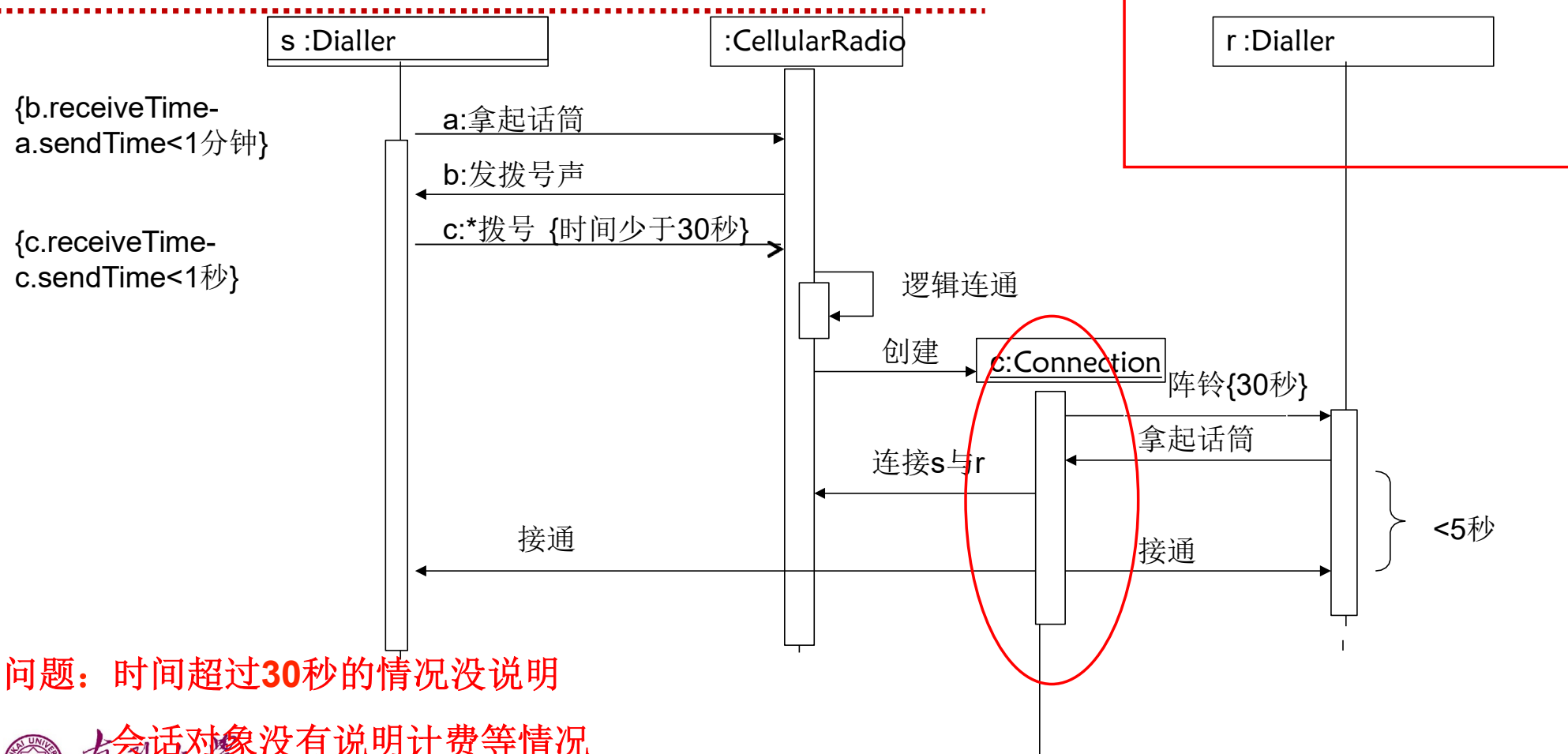


思考题：有缺陷的顺序图

- 这个顺序图有什么问题？
(留意UML的语法以及场景的可行性)



修改方案一



问题：时间超过**30秒**的情况没说明

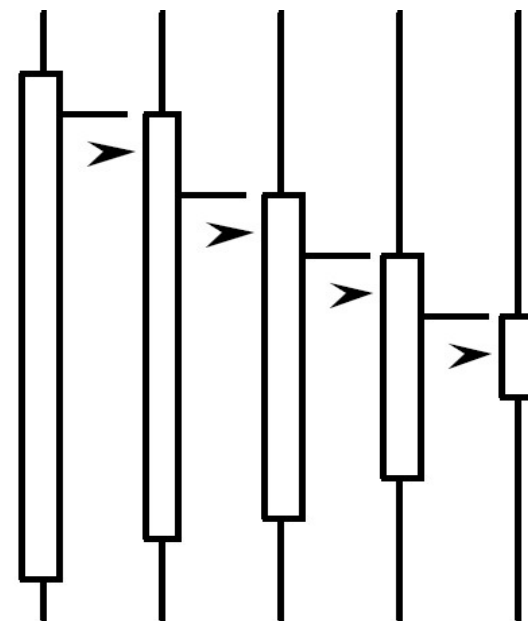
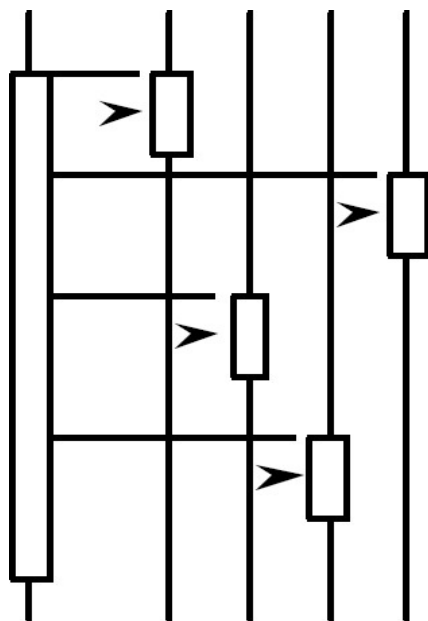
会话对象没有说明计费等情况



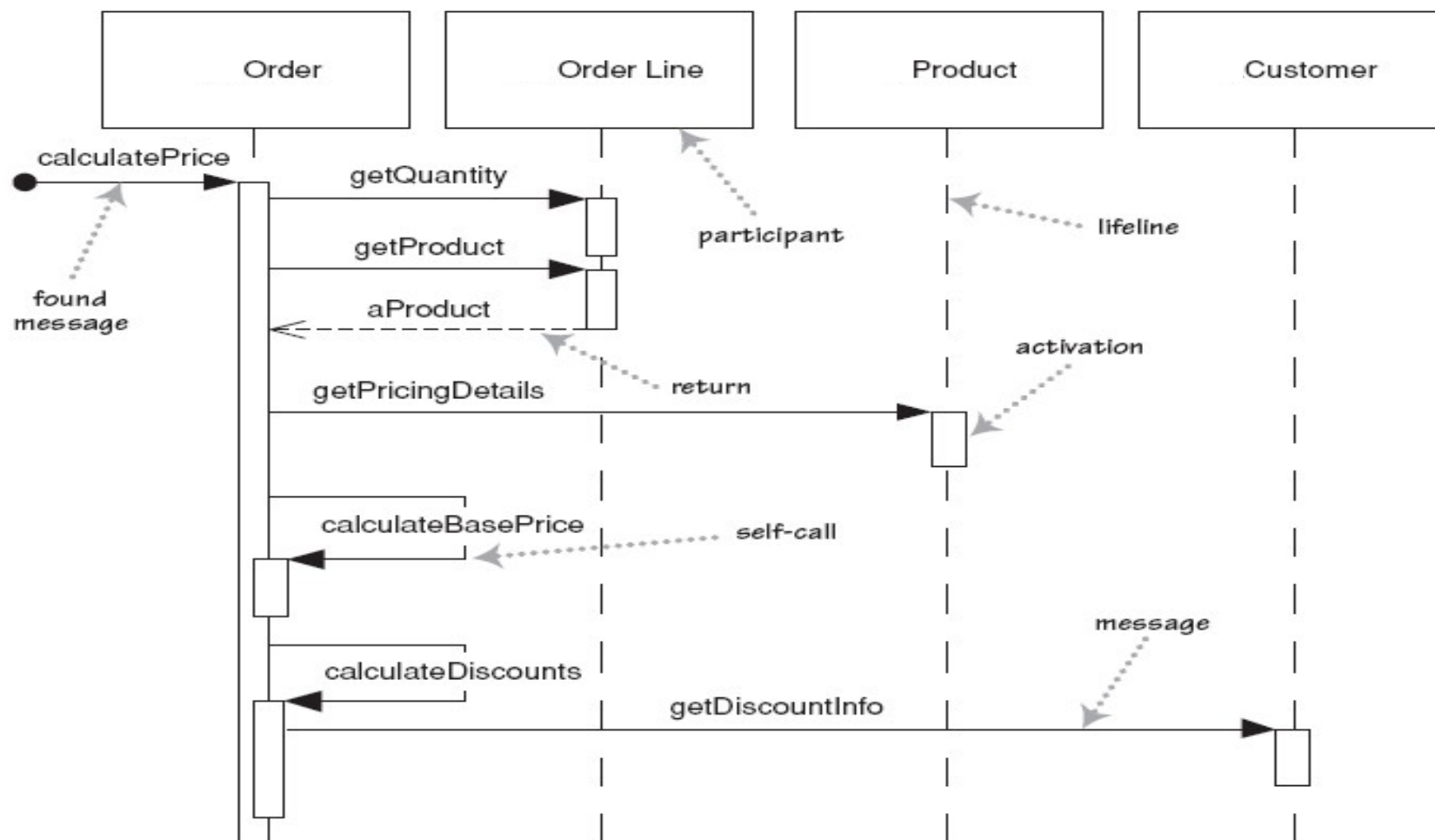
集中、分布控制

• 下述两种系统的控制流有什么特点？

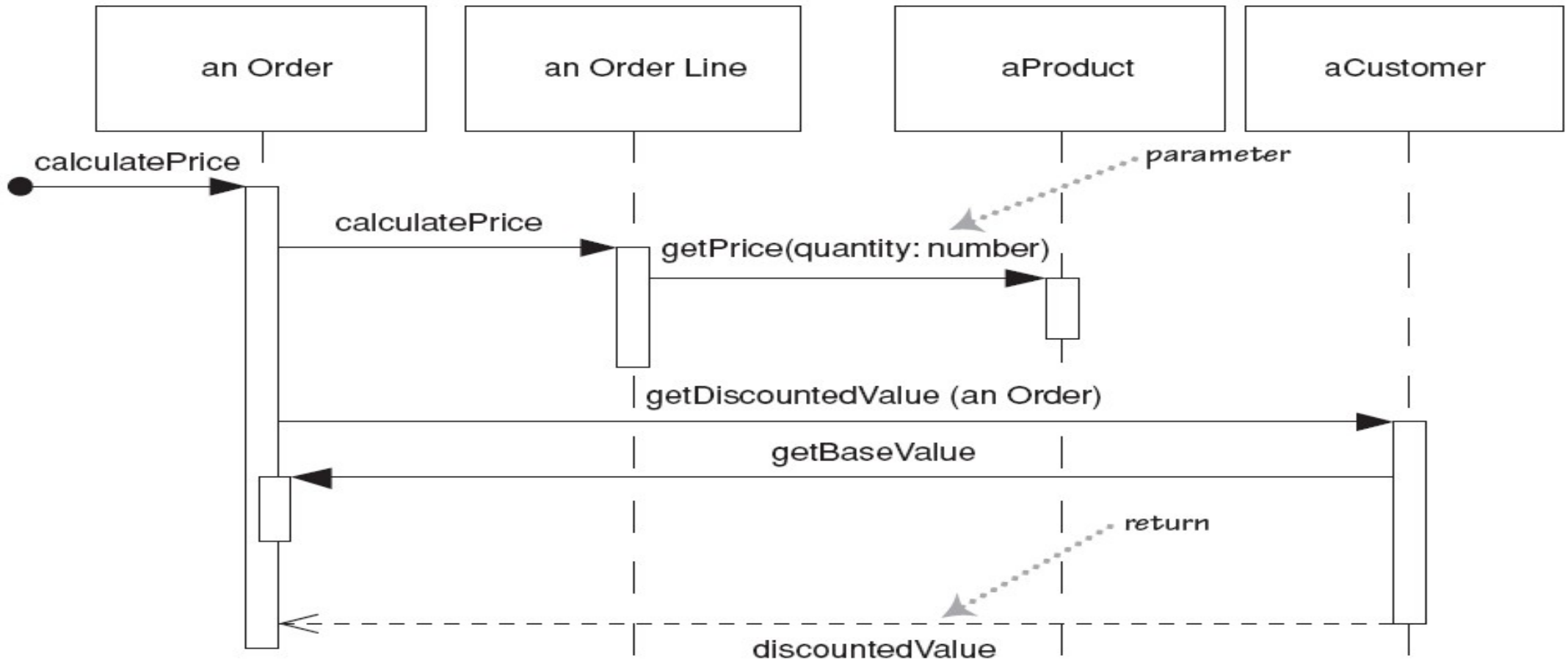
- 集中式的？
- 分布式的？



例1：集中控制的计价系统顺序图



例2：分布控制的计价系统顺序图

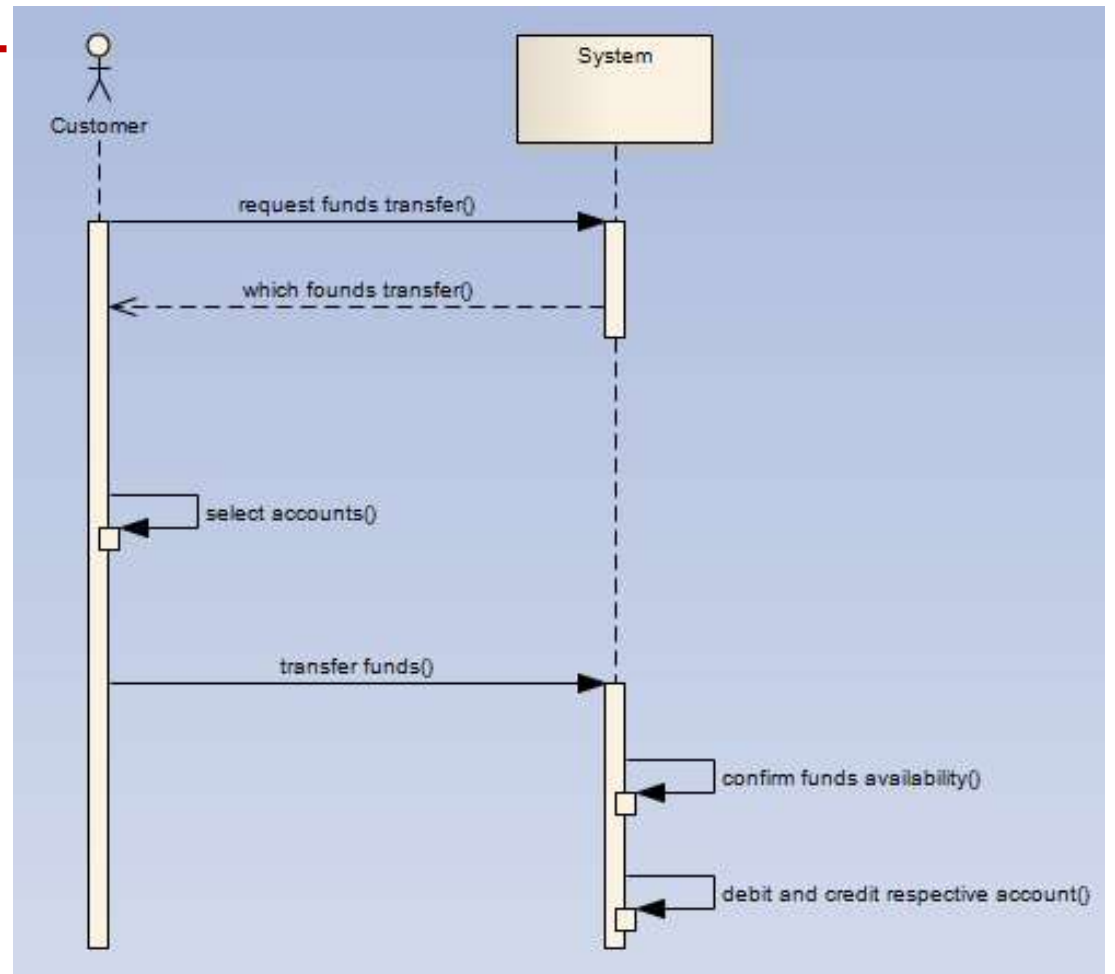


顺序图与用例的关系 I

- 顺序图表达**单个情景**实例的行为。
- 每个用例对应一个顺序图。
- 顺序图表达对象间如何**协作**完成用例所描述的功能。
- 顺序图用于表示为完成用例而在系统边界输入输出的**数据**以及**消息**
- 顺序图也用于表示系统内部对象间的消息传递。

顺序图与用例对应

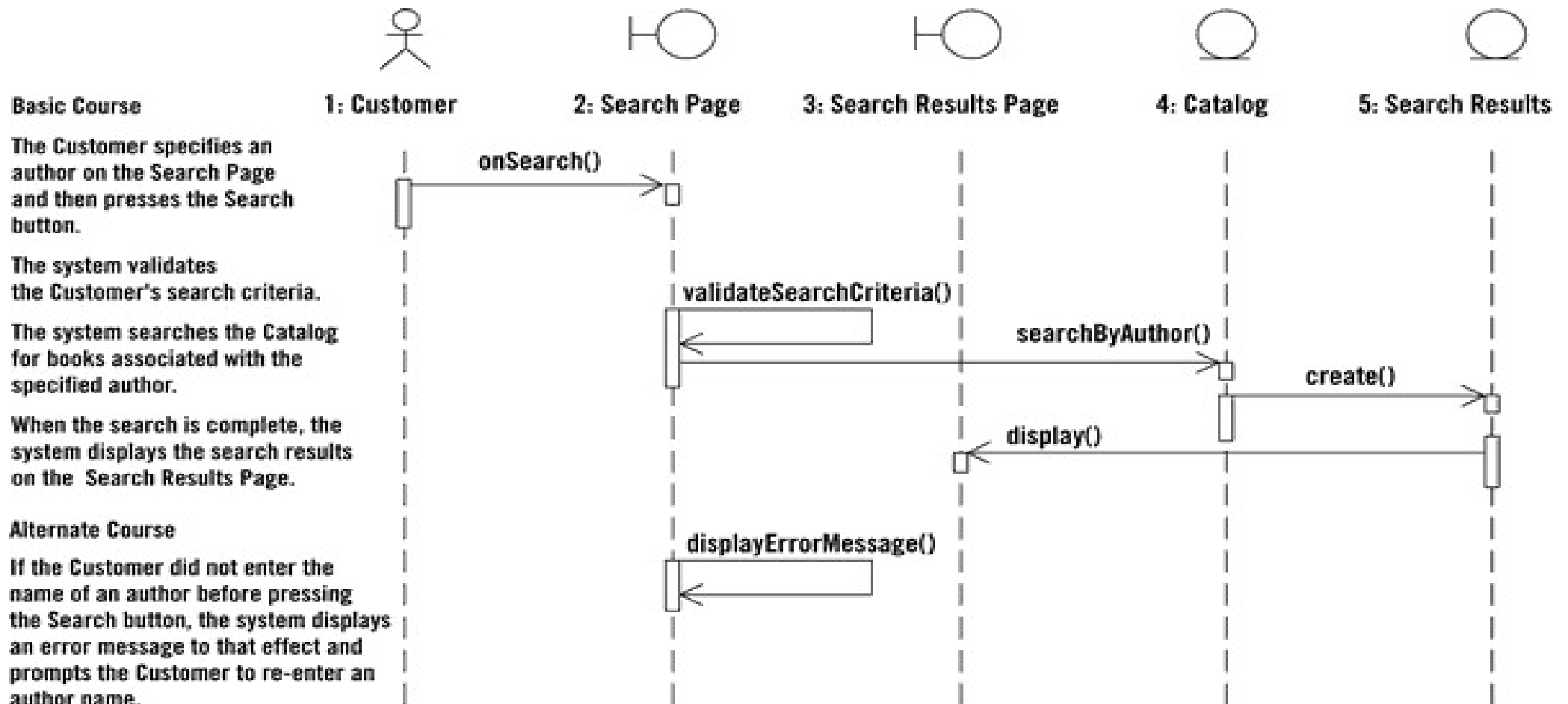
1. The customer requests a funds transfer.
2. The system asks the user to identify the accounts between which funds are to be transferred and the transfer amount.
3. Customer selects the account to transfer funds from, the account to transfer to, and then indicates the amount of funds to transfer.
4. The system checks the account from which funds are to be transferred and confirms that sufficient funds are available.
5. The amount is debited to the account from which funds are to be transferred and credited to the account previously selected by the customer by the system.



顺序图与用例的关系 II

- 顺序图可帮助分析人员对用例图进行扩展、细化和补遗
- 顺序图可用于开发周期的不同阶段，服务于不同目的，描述不同粒度的行为
- 分析阶段的顺序图**不要**
 - 包含设计对象
 - 关注消息参数

从用例中抽取顺序图



顺序图建模意义

- 通过顺序图描述算法逻辑
- 高质量的顺序图是代码的抽象
- 顺序图是与语言无关的表示方式
- 可以绘制顺序图来描述业务逻辑
- 可以通过团队协作完成顺序图的绘制
- 可以在同一页浏览多个对象和类的行为

顺序图建模风格

- 建模风格1：把注意力集中于关键的交互。
 - 创建模型时要把注意力集中于系统的关键方面，而不要包括无关的细节。
 - 如果顺序图是用于描述业务逻辑的，就没必要包括对象和数据库之间的详细交互。

顺序图建模风格

- 建模风格2：对于参数，优先考虑使用参数名而不是参数类型。

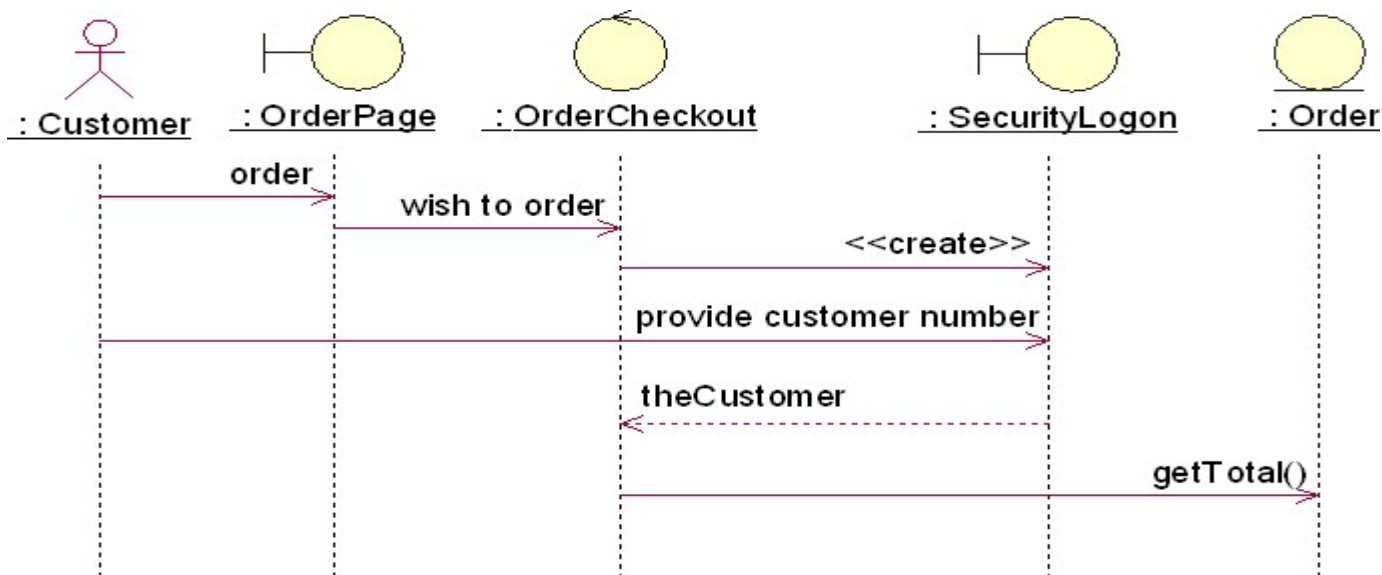
- 例如

消息 `addDeposit(amount, target)` 比 `addDeposit(Currency, Account)` 传递了更多的信息

- 在消息中只使用类型信息不能传递足够的信息
 - 参数的类型信息用UML类图表示更好

顺序图建模风格

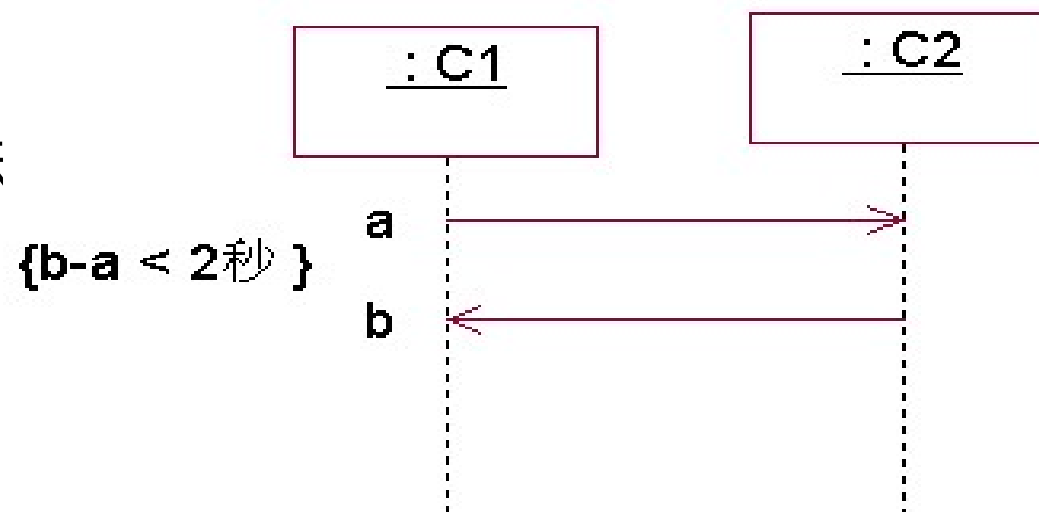
- 建模风格3：不要对明显的返回值建模。



- 建模风格4：可以把返回值建模为方法调用的一部分。

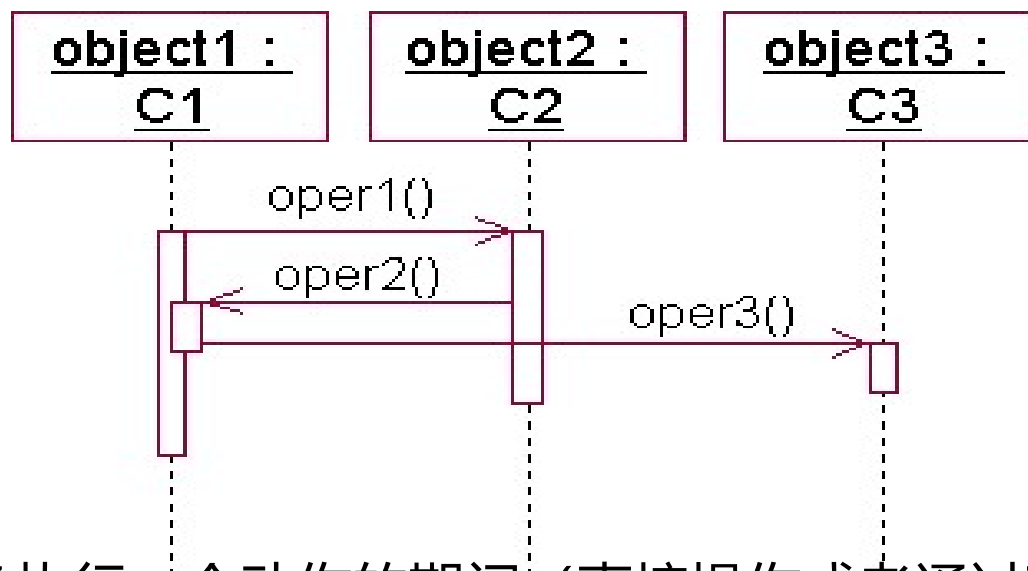
顺序图常见问题分析

- 顺序图中时间约束的表示
 - 用约束 (constraint)来表示



控制焦点(focus of control)的嵌套

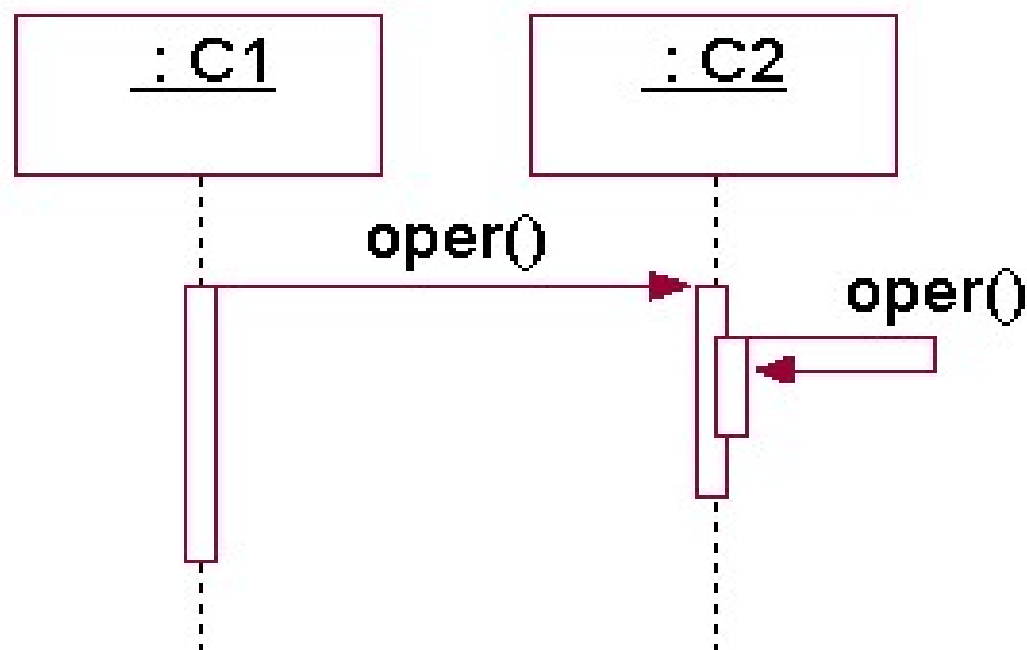
- 嵌套的FOC可以更精确地说明消息的开始和结束位置。
- 图例：



激活期(activation)：表示对象执行一个动作的期间（直接操作或者通过下级操作），也即对象激活的时间段。
控制焦点和激活期是同一个概念。

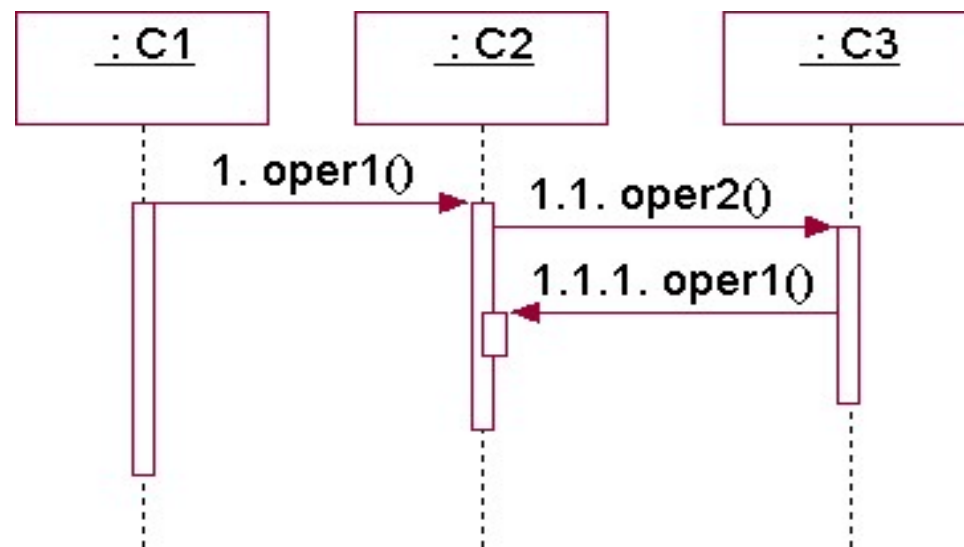
顺序图常见问题分析

- 顺序图中递归的表示
- 利用嵌套的FOC表示
- 例1. 单个对象自身的递归。



顺序图常见问题分析

- 顺序图中递归的表示
 - 利用嵌套的FOC表示
- 例2. 多个对象间相互递归调用的表示。



顺序图的作用

- 帮助分析人员对照检查用例中描述需求，是否已经落实给具体对象去实现
- 提醒分析人员去补充遗漏的对象类或操作
- 帮助分析人员识别哪些对象是主动对象
- 通过对一个特定的对象群体的动态方面建模，深入地理解对象之间的交互

顺序图中带条件消息的发送，不能采用以下哪种方式进行？

提交

- A 用文字说明，作为注释添加
- B 添加条件控制框
- C 分成多个顺序图子图来描述
- D 在消息名字前加条件子句

顺序图的组合控制框 (Frame) 中, 用于表达分支选择关系的控制符是哪个?

提交

- A opt
- B alt
- C ref
- D loop



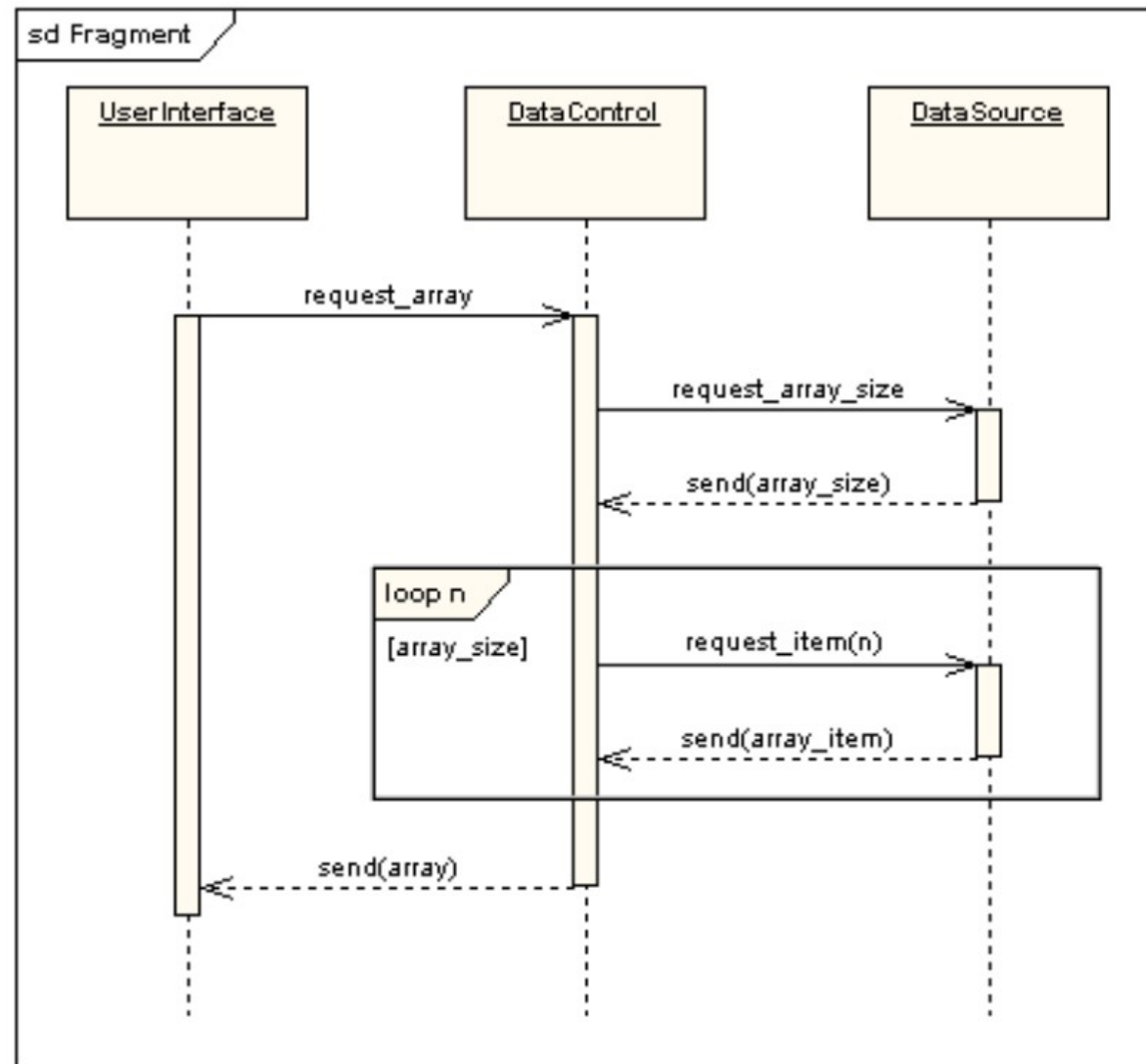
当一个顺序图过大时，最好的处理方法是？

提交

- A 添加注释，说明顺序图的各部分之间的接续关系
- B 用不完整的箭头，指明本页的顺序图未完待续
- C 去掉无关的细节，保持图的简洁性
- D 添加ref框，建立顺序图间的引用关系

对顺序图与用例之间关系的阐述，错误的是：

- A 顺序图表达单个情景实例的行为，每个用例对应一个顺序图
- B 用例分析阶段的顺序图要包含设计对象，并关注消息参数
- C 顺序图用于表示为完成用例而在系统边界输入输出的数据以及消息
- D 顺序图可帮助分析人员对用例图进行扩展、细化和补遗



包含哪个结构？

它的控制逻辑
是什么？



该顺序图中包含哪个结构?

提交

- A 分支选择结构
- B 并发结构
- C 单条件结构
- D 循环结构

以下哪个说法是正确的？

提交

- A 对于参数，优先考虑使用参数类型而不是参数名
- B 每个用例对应一个顺序图
- C 在顺序图过程中不能创建对象
- D 顺序图应该用于设计阶段而不是需求分析阶段

